

```
==== src/redmail/_main_.py (147 строк) ====
1  from __future__ import annotations

3  import subprocess
4  import os
5  import sys
6  from importlib.metadata import PackageNotFoundError, version

8  from PySide6.QtCore import Qt
9  from PySide6.QtGui import QColor, QFont, QPainter, QPixmap
10 from PySide6.QtWidgets import QApplication, QSplashScreen

12 from redmail import applog
13 from redmail import profile
14 from redmail import config_store
15 from redmail.ui import theme

17 _SPLASH_SIZE = (420, 240)
18 _SPLASH_BG = "#1a73e8"
19 _SPLASH_FG = "white"

22 def app_version() -> str:
23     """Версия для заставки и "О программе" – жалоба: "добавь в справку о
24     программе версию, сейчас там 0.0.1", т.е. одна и та же версия пакета
25     для КАЖДОЙ сборки (в pyproject.toml она не меняется между RPM-релизами,
26     там всегда "0.0.1" – реальный номер сборки живёт только в Release: у
27     .спец). На RED OS сначала спрашиваем сам установленный RPM-пакет
28     (даёт "0.0.1-30", ровно то, что нужно, чтобы отличить сборки друг от
29     друга); вне RED OS (разработка, другой дистрибутив) rpm просто нет –
30     тогда версия пакета Python как раньше."""
31     try:
32         result = subprocess.run(
33             ["rpm", "-q", "--queryformat", "%{VERSION}-%{RELEASE}", "redmail"],
34             capture_output=True,
35             text=True,
36             timeout=2,
37         )
38         if result.returncode == 0 and result.stdout.strip():
39             return result.stdout.strip()
40     except (OSError, subprocess.SubprocessError):
41         pass
42     try:
43         return version("redmail")
44     except PackageNotFoundError:
45         return "?"

48 def build_splash_pixmap(version: str) -> QPixmap:
49     """Заставка при запуске (жалоба: "открывай сразу информационное окно
50     до открытия основного окна... название, автора и ход загрузки") –
51     название/версия/автор совпадают с тем, что уже показывает "О
52     программе...", чтобы не заводить два разных источника правды."""
53     width, height = _SPLASH_SIZE
54     pixmap = QPixmap(width, height)
55     pixmap.fill(QColor(_SPLASH_BG))
56     painter = QPainter(pixmap)
57     painter.setRenderHint(QPainter.RenderHint.Antialiasing)
58     painter.setPen(QColor(_SPLASH_FG))

60     title_font = QFont()
61     title_font.setPointSize(22)
62     title_font.setBold(True)
63     painter.setFont(title_font)
64     painter.drawText(pixmap.rect().adjusted(24, 28, -24, 0), Qt.AlignmentFlag.AlignLeft |
Qt.AlignmentFlag.AlignTop, "RedMail")

66     text_font = QFont()
67     text_font.setPointSize(11)
68     painter.setFont(text_font)
69     lines = (
70         "Почтовый клиент для RED OS",
71         f"Версия {version}",
72         "Автор: Пономарев Роман Сергеевич",
73     )
74     for i, line in enumerate(lines):
75         painter.drawText(
76             pixmap.rect().adjusted(24, 74 + i * 22, -24, 0),
77             Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignTop,
78             line,
79         )
80     painter.end()
81     return pixmap

84 def _ensure_session_bus_address() -> None:
```

```

85     """Запуск не из сессии рабочего стола (ssh, автозапуск до экспорта
86     переменных) — без DBUS_SESSION_BUS_ADDRESS keyring не найдёт
87     Secret Service и решит, что хранилища паролей нет. Стандартный адрес
88     шины пользователя — $XDG_RUNTIME_DIR/bus; если файл есть, подставляем."""
89     if os.environ.get("DBUS_SESSION_BUS_ADDRESS"):
90         return
91     runtime_dir = os.environ.get("XDG_RUNTIME_DIR") or f"/run/user/{os.getuid()}" if hasattr(os, "getuid") else ""
92     if runtime_dir and os.path.exists(os.path.join(runtime_dir, "bus")):
93         os.environ["DBUS_SESSION_BUS_ADDRESS"] = "unix:path=" + os.path.join(runtime_dir, "bus")

96 def main() -> int:
97     log_file = applog.setup_logging()
98     _ensure_session_bus_address()
99     applog.get_logger("app").info("Запуск программы (журнал: %s)", log_file)
100    app = QApplication(sys.argv)

102    # Жалоба (и после первой правки с repaint()): "информационное окно
103    # выводится не сразу, долго висит, потом появляется, и практически
104    # мгновенно открывается окно приложения" — то есть задержка была ДО
105    # показа заставки, а не после. Две причины: (1) `from
106    # redmail.ui.main_window import MainWindow` стоял на уровне модуля и
107    # тянул за собой PySide6.QtWebEngine* — инициализация Chromium занимает
108    # секунды ещё до входа в main(); теперь импорт отложен и сам является
109    # шагом "Загрузка интерфейса..." уже при видимой заставке; (2)
110    # app_version() запускает `rpm -q` (до 2 с на медленной VM) — тоже до
111    # показа. Заставка показывается сразу с версией "...", версия
112    # дорисовывается следом. repaint() — синхронная перерисовка: одного
113    # processEvents() недостаточно, чтобы окно гарантированно оказалось
114    # на экране до долгой блокировки потока (особенно X11 без композитора).
115    splash = QSplashScreen(build_splash_pixmap(""))
116    splash.show()
117    splash.repaint()
118    app.processEvents()

120    def report(message: str) -> None:
121        splash.showMessage(
122            message, Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignBottom, QColor(_SPLASH_FG)
123        )
124        splash.repaint()
125        app.processEvents()

127    version = app_version()
128    applog.get_logger("app").info("Версия %s", version)
129    splash.setPixmap(build_splash_pixmap(version))
130    report("Применение темы оформления...")
131    theme.apply_theme(app, config_store.load_theme())

133    report("Загрузка интерфейса...")
134    from redmail.ui.main_window import MainWindow

136    profile_path = profile.ensure_profile()
137    applog.get_logger("app").info("Профиль: %s", profile_path)
138    window = MainWindow()

140    report("Готово")
141    window.show()
142    splash.finish(window)
143    return app.exec()

146 if __name__ == "__main__":
147     raise SystemExit(main())

```

==== src/redmail/imap_client.py (948 строк) ====

```

1  from __future__ import annotations

3  import functools
4  import imaplib
5  import threading
6  from dataclasses import dataclass, field
7  from email import message_from_bytes
8  from email.header import Header, decode_header
9  from email.message import Message

11 from imapclient import IMAPClient
12 from imapclient.exceptions import IMAPClientError

14 from redmail.applog import get_logger

16 _log = get_logger("imap")

18 _HEADER_FIELDS = "BODY.PEEK[HEADER.FIELDS (IMPORTANCE X-PRIORITY)]"

20 # Флаг \Flagged ставим всегда вместе с цветом — так другие IMAP-клиенты
21 # (Thunderbird, сам Outlook по IMAP) увидят письмо помеченным, даже если не
22 # понимают наш собственный keyword с цветом. $-префикс — общепринятое
23 # соглашение для нестандартных keyword-флагов (как $Forwarded, $MDNSent).
24 MARKER_COLORS: dict[str, bytes] = {

```

```

25     "red": b"$RedMailRed",
26     "orange": b"$RedMailOrange",
27     "yellow": b"$RedMailYellow",
28     "green": b"$RedMailGreen",
29     "blue": b"$RedMailBlue",
30     "purple": b"$RedMailPurple",
31 }
32 _COLOR_BY_KEYWORD = {v: k for k, v in MARKER_COLORS.items()}

35 def split_markers(value: str | None) -> list[str]:
36     """Несколько маркеров на письме (пожелание: "на письмо можно поставить
37     несколько маркеров") хранятся в том же поле marker_color через запятую
38     ("red,blue") — так не меняются схема кэша/архива и все места, где это
39     поле просто прокидывается дальше; разбирают его только те, кому нужны
40     отдельные цвета (иконка, фильтр, меню, IMAP-флаги)."""
41     return [color for color in (value or "").split(",") if color]

44 def join_markers(colors) -> str | None:
45     ordered: list[str] = []
46     for color in colors:
47         if color and color not in ordered:
48             ordered.append(color)
49     return ",".join(ordered) or None

51 # Таймаут одной операции на сожете IMAP (секунды). Не ограничивает
52 # длительность загрузки большого письма целиком — только паузу между
53 # порциями данных от сервера.
54 SOCKET_TIMEOUT = 120

56 # Сентинел по умолчанию для set_marker(previous_color=...) — отличает "вызывающий
57 # код не знает текущий маркер" (безопасный медленный путь: снять все
58 # возможные keyword'ы) от "previous_color=None" (точно знает, что маркера не
59 # было — снимать нечего). Спутать их означало бы на реальном сервере либо
60 # лишние round trip'ы, либо оставленный висеть старый keyword.
61 UNKNOWN_MARKER = object()

64 def _is_recoverable_by_reconnect(exc: Exception) -> bool:
65     """Протокольные ошибки (imaplib.IMAP4.error = IMAPClientError: команда
66     сервером понята, но отвергнута), которые всё же лечатся
67     переподключением — в отличие от остальных, см. _reconnecting:

69     * "command SELECT/... illegal in state NONAUTH" — сервер молча
70     разлогинил сессию после долгого простоя (жалоба: "после долгого
71     простоя выдаёт... лечится перезапуском"), но CAM TCP-сокет при этом
72     мог и не порваться, поэтому это не OSError/EOFError/IMAP4.abort;
73     * "[UNAVAILABLE] Failed to open mailbox" / "Service temporarily
74     unavailable" (RFC 5530: временный отказ подсистемы сервера — жалоба:
75     "после сбоя сервера или принудительного простоя не восстанавливается
76     подключение, обновление даёт ошибку") — сессия после такого сбоя
77     сервера обычно уже невалидна, свежее соединение проходит."""
78     text = str(exc)
79     if "illegal in state" in text and "NONAUTH" in text:
80         return True
81     return "[UNAVAILABLE]" in text.upper()

84 def _reconnecting(method):
85     """После простоя реальный IMAP-сервер молча рвёт TCP-соединение (никто
86     не обязан держать сессию вечно — RFC 3501 не гарантирует этого), и
87     следующая же операция падала с сырой сетевой ошибкой (жалоба
88     пользователя: "после простоя часто выдаёт ошибку подключения").
89     OSError — общий предок и для обрыва соединения, и для TLS-ошибок в
90     современном Python.

92     imaplib.IMAP4.abort — ОТДЕЛЬНО от OSError, хотя семантически это тот
93     же случай: сам imaplib документирует его как "Service errors - close
94     and retry" (imaplib.py), и на практике так оборачивает разрыв TLS
95     ("EOF occurred in violation of protocol") при чтении строки ответа —
96     `class error(Exception)` в стандартной библиотеке НЕ наследуется от
97     OSError, так что раньше это вообще не попадало под переподключение
98     (жалоба: "периодически выдаёт ошибки" при обновлении/чтении письма —
99     ошибка показывалась как есть с первого же раза, без единой попытки
100    восстановить соединение). Настоящие протокольные ошибки
101    (IMAPClientError на команду, которую сервер понял, но отверг) НЕ
102    перехватываются — переподключение их не лечит, показываем как есть,
103    КРОМЕ "illegal in state NONAUTH" — см. _is_stale_session_error."""

105    @functools.wraps(method)
106    def wrapper(self, *args, **kwargs):
107        # Одно соединение — одна команда за раз. Тело письма грузится в
108        # фоновом потоке, а отметка "прочитано"/маркер/следующий fetch
109        # могли уйти в тот же сокет из другого потока параллельно; imaplib
110        # к этому не готов — ответы перемешиваются, и один из потоков ждёт
111        # своего ответа вечно (жалоба: "подвисает при переходе от письма к
112        # письму на загрузке письма"). RLock — потому что декорированные
113        # методы вызывают друг друга.

```

```

114         with self._lock:
115             try:
116                 return method(self, *args, **kwargs)
117             except (OSError, EOFError, imaplib.IMAP4.abort) as exc:
118                 _log.warning("IMAP %s: %s - обрыв соединения (%s), переподключение", self.account.host,
method.__name__, exc)
119             try:
120                 self._reconnect()
121             except Exception as reconnect_exc:
122                 _log.error("IMAP %s: переподключение не удалось: %s", self.account.host, reconnect_exc)
123                 raise exc from None # переподключиться тоже не вышло - исходная ошибка нагляднее
124                 return method(self, *args, **kwargs)
125             except imaplib.IMAP4.error as exc:
126                 if not _is_recoverable_by_reconnect(exc):
127                     _log.error("IMAP %s: %s - ошибка протокола: %s", self.account.host, method.__name__, exc)
128                     raise
129                 _log.warning("IMAP %s: %s - сессия недействительна (%s), переподключение", self.account.host,
method.__name__, exc)
130             try:
131                 self._reconnect()
132             except Exception as reconnect_exc:
133                 _log.error("IMAP %s: переподключение не удалось: %s", self.account.host, reconnect_exc)
134                 raise exc from None
135                 return method(self, *args, **kwargs)
137         return wrapper

140 @dataclass
141 class Account:
142     host: str
143     username: str
144     password: str
145     port: int = 993
146     use_ssl: bool = True
147     # "password" - обычный LOGIN; "kerberos" - SSO для почтового сервера в
148     # домене: аутентификация идёт по Kerberos-билету, который ОС уже
149     # выдала при входе пользователя в домен (RED OS + SSSD), пароль в
150     # приложении не хранится и не используется (см. gssapi_sasl.py).
151     auth_type: str = "password"
152     # Необязательный keytab как источник билета для SSO (вместо билета из
153     # системного кэша) и principal, для которого он выписан - см.
154     # gssapi_sasl.acquire_credentials.
155     keytab_path: str = ""
156     principal: str = ""

159 @dataclass
160 class FolderInfo:
161     name: str
162     delimiter: str

165 @dataclass
166 class MessageSummary:
167     uid: int
168     subject: str
169     sender: str
170     sender_email: str
171     date: str
172     message_id: str
173     has_attachments: bool = False
174     marker_color: str | None = None
175     importance: str = "normal" # "high" | "normal" | "low"
176     is_read: bool = False
177     # Только для списка "Отправленные" - там "От кого" всегда сам
178     # пользователь, бесполезная колонка (жалоба: "в отправленных нет поля
179     # адресат, невозможно понять кому писали"). Достаётся бесплатно - те же
180     # данные ENVELOPE, что уже фетчатся для sender, просто ещё одно поле.
181     to: str = ""
182     # Стандартный IMAP-флаг \Answered - раньше нигде не читался и не
183     # проставлялся (жалоба: "если мы ответили на письмо, это никак не
184     # отражается, нужен какой-то признак").
185     is_answered: bool = False
186     # RFC822.SIZE - по нему фоновая синхронизация решает, качать ли тело
187     # сразу или отложить до открытия (порог «вложения > 25 МБ по запросу»).
188     size: int = 0

191 @dataclass
192 class Attachment:
193     filename: str
194     content_type: str
195     payload: bytes

197     @property
198     def size(self) -> int:
199         return len(self.payload)

```

```

202 @dataclass
203 class MessageContent:
204     text: str
205     attachments: list[Attachment] = field(default_factory=list)
206     html: str = ""
207     # Content-Id (без угловых скобок) -> (content_type, данные) - картинки,
208     # встроенные в HTML через , а не обычные вложения.
209     inline_images: dict[str, tuple[str, bytes]] = field(default_factory=dict)
210     # Реквизиты письма (тема/отправитель/получатели) - раньше нигде не
211     # показывались при просмотре письма (жалоба: "невидно его реквизитов
212     # (тема, отправитель, адресаты)"). MessageSummary уже даёт subject/
213     # sender для списка писем, но не даёт To/Сс - они здесь.
214     subject: str = ""
215     from_: str = ""
216     to: str = ""
217     cc: str = ""
218     bcc: str = ""

221 class ImapSession:
222     """Одно живое IMAP-соединение на всё время работы с ящиком.

224     Открывать новое соединение (TCP + TLS + логин) на каждый клик по папке
225     или письму - секунды задержки на медленной сети. Здесь соединение
226     держится, пока пользователь не переподключится или не закроет окно.
227     """

229     def __init__(self, account: Account):
230         self.account = account
231         self._lock = threading.RLock()
232         self._client = self._new_client()
233         try:
234             self._login()
235         except Exception as exc:
236             _log.error("IMAP %s:%s: вход не удался (%s, %s): %s", account.host, account.port, account.username,
account.auth_type, exc)
237             raise
238         self._selected_folder: str | None = None
239         self._selected_exists = 0
240         self._raw_folders: list[tuple] = []

242     def _login(self) -> None:
243         if self.account.auth_type == "kerberos":
244             # Импорт внутри функции: пакет gssapi требует системных
245             # библиотек Kerberos, которых нет на части машин (Windows,
246             # окружения без домена) - обычный пароль не должен ломаться
247             # из-за отсутствия зависимости, нужной только для SSO.
248             from redmail import gssapi_sasl

250             gssapi_sasl.imap_sasl_login(
251                 self._client,
252                 self.account.host,
253                 self.account.username,
254                 keytab_path=self.account.keytab_path,
255                 principal=self.account.principal,
256             )
257         else:
258             self._client.login(self.account.username, self.account.password)
259             _log.info("IMAP %s:%s: вход выполнен (%s, %s)", self.account.host, self.account.port,
self.account.username, self.account.auth_type)

261     def _new_client(self) -> IMAPClient:
262         # Таймаут на сокетe обязателен: без него любое зависшее чтение
263         # (сервер молча перестал отвечать, рассинхрон ответов) блокирует
264         # поток навсегда - с таймаутом это OSError, которую _reconnecting
265         # лечит переподключением и повтором.
266         return IMAPClient(self.account.host, port=self.account.port, ssl=self.account.use_ssl,
timeout=SOCKET_TIMEOUT)

268     def close(self) -> None:
269         with self._lock:
270             try:
271                 self._client.logout()
272             except Exception:
273                 pass

275     def _reconnect(self) -> None:
276         self._client = self._new_client()
277         try:
278             self._login()
279         except Exception as exc:
280             _log.error("IMAP %s: повторный вход не удался: %s", self.account.host, exc)
281             raise
282         if self._selected_folder is not None:
283             # Кое-что из вызывающего кода (fetch_summaries) не делает
284             # собственный SELECT - предполагается, что папка уже выбрана
285             # предыдущим folder_message_count()/_select(). Восстанавливаем
286             # это состояние сразу, иначе повтор упал бы снова, уже по

```

```

287         # другой причине (ничего не выбрано).
288         self._client.select_folder(self._selected_folder, readonly=False)

290     def __enter__(self) -> "ImapSession":
291         return self

293     def __exit__(self, *exc_info) -> None:
294         self.close()

296     @_reconnecting
297     def list_folders(self) -> list[FolderInfo]:
298         # Сырой ответ запоминаем — из него же достаём папку "Корзина" в
299         # trash_folder(), без второго похода на сервер (find_special_folder
300         # библиотеки сам заново вызывает list_folders).
301         self._raw_folders = self._client.list_folders()
302         return [
303             FolderInfo(name=name, delimiter=(delimiter or b"/").decode("ascii", errors="replace"))
304             for flags, delimiter, name in self._raw_folders
305             if b"\Noselect" not in flags
306         ]

308     @_reconnecting
309     def create_folder(self, name: str) -> None:
310         self._client.create_folder(name)

312     @_reconnecting
313     def rename_folder(self, old_name: str, new_name: str) -> None:
314         self._client.rename_folder(old_name, new_name)

316     def trash_folder(self) -> str | None:
317         return self._special_folder(b"\Trash", ("trash", "корзин"))

319     def sent_folder(self) -> str | None:
320         return self._special_folder(b"\Sent", ("sent", "отправленн"))

322     def drafts_folder(self) -> str | None:
323         return self._special_folder(b"\Drafts", ("draft", "черновик"))

325     def _special_folder(self, special_use_flag: bytes, name_hints: tuple[str, ...]) -> str | None:
326         # Сначала — SPECIAL-USE (RFC 6154), это надёжно, сервер сам сказал,
327         # какая папка какая. Но объявляет его не каждый реальный сервер —
328         # без запасного варианта по имени такая папка оставалась вовсе не
329         # распознанной: письмо можно было открыть, но не отправить (жалоба
330         # "из черновика не даёт отправить" — двойной клик по письму в
331         # "Черновиках" тихо считал, что это просто обычная папка, и
332         # открывал письмо на просмотр, а не на редактирование).
333         for flags, _delimiter, name in self._raw_folders:
334             if special_use_flag in flags:
335                 return name
336         for _flags, _delimiter, name in self._raw_folders:
337             lowered = name.lower()
338             if any(hint in lowered for hint in name_hints):
339                 return name
340         return None

342     @_reconnecting
343     def append_message(self, folder: str, raw: bytes, *, flags: tuple[bytes, ...] = ()) -> None:
344         """Кладёт готовое (уже собранное) сообщение в папку напрямую,
345         минуя SMTP — для "Отправленные" (сервер сам не всегда сохраняет
346         копию исходящих) и "Черновики" (письмо, которое никуда не
347         отправлялось)."""
348         self._client.append(folder, raw, flags=flags)

350     @_reconnecting
351     def folder_message_count(self, folder: str) -> int:
352         """SELECT папку, вернуть общее число писем в ней (EXISTS).

354         Дешёвая операция (без сканирования, в отличие от SEARCH) — на ней
355         удобно проверять, изменилось ли что-то в папке с прошлого раза,
356         прежде чем платить за полный FETCH сводок (см. CachedMailbox).
357         """
358         status = self._client.select_folder(folder, readonly=False)
359         self._selected_folder = folder
360         self._selected_exists = status[b"EXISTS"]
361         return self._selected_exists

363     @_reconnecting
364     def folder_unseen_count(self, folder: str) -> int:
365         """Число непочитанных писем в папке — жалоба: "подписывать
366         количество писем" рядом с папкой в дереве. STATUS, а не
367         select_folder/SEARCH: не переключает "текущую" папку сессии
368         (folder_message_count это делает) и не сканирует письма — можно
369         дёшево опросить сразу много папок подряд при построении дерева."""
370         status = self._client.folder_status(folder, ["UNSEEN"])
371         for key, value in status.items():
372             key_name = key.decode("ascii", errors="replace") if isinstance(key, bytes) else str(key)
373             if key_name.upper() == "UNSEEN":
374                 return int(value)
375         return 0

```

```

377     @_reconnecting
378     def fetch_summaries(self, limit: int = 50) -> list[MessageSummary]:
379         """Сводки последних `limit` писем уже выбранной папки.
380
381         Требуется, чтобы перед этим была вызвана folder_message_count -
382         отдельного SELECT здесь больше нет.
383         """
384         total = self._selected_exists
385         if total == 0:
386             return []
387         start = max(1, total - limit + 1)
388
389         # Порядковые номера, а не UID - иначе пришлось бы всё равно узнавать
390         # реальные UID через SEARCH. UID запрашиваем отдельным полем: он
391         # возвращается независимо от режима нумерации.
392         self._client.use_uid = False
393         try:
394             response = self._client.fetch(
395                 f"{start}:*", ["ENVELOPE", "UID", "FLAGS", "BODYSTRUCTURE", _HEADER_FIELDS]
396             )
397         finally:
398             self._client.use_uid = True
399
400         by_seq = sorted(response.items(), key=lambda item: item[0], reverse=True)
401         return [_to_summary(data) for _seq, data in by_seq]
402
403     def fetch_folder_summaries(self, folder: str = "INBOX", limit: int = 50) -> list[MessageSummary]:
404         self.folder_message_count(folder)
405         return self.fetch_summaries(limit)
406
407     @_reconnecting
408     def folder_status(self, folder: str) -> tuple[int, int]:
409         """(UIDVALIDITY, число писем) через STATUS - без SELECT и без
410         сканирования; UIDVALIDITY нужен полной синхронизации: если сервер
411         перевыдал UID, локальную копию папки надо перестроить."""
412         status = self._client.folder_status(folder, ["UIDVALIDITY", "MESSAGES"])
413         validity = 0
414         messages = 0
415         for key, value in status.items():
416             name = key.decode("ascii", errors="replace") if isinstance(key, bytes) else str(key)
417             if name.upper() == "UIDVALIDITY":
418                 validity = int(value)
419             elif name.upper() == "MESSAGES":
420                 messages = int(value)
421         return validity, messages
422
423     @_reconnecting
424     def fetch_summaries_by_uids(self, folder: str, uids: list[int]) -> list[MessageSummary]:
425         """Сводки конкретных писем по UID (полная синхронизация качает
426         заголовки порциями, от новых к старым). RFC822.SIZE - чтобы решить,
427         качать ли тело сразу."""
428         if not uids:
429             return []
430         self._select(folder)
431         response = self._client.fetch(
432             list(uids), ["ENVELOPE", "UID", "FLAGS", "BODYSTRUCTURE", "RFC822.SIZE", _HEADER_FIELDS]
433         )
434         summaries = []
435         for key, data in sorted(response.items(), key=lambda item: item[0], reverse=True):
436             if b"ENVELOPE" not in data:
437                 continue
438             # В UID-режиме imapclient ключом ответа и есть UID, а поле UID в
439             # данных может отсутствовать (реальный Dovecot) - подставляем.
440             data.setdefault(b"UID", key)
441             summaries.append(_to_summary(data))
442         return summaries
443
444     @_reconnecting
445     def fetch_flags(self, folder: str, uids: list[int]) -> dict[int, tuple[bool, bool, str | None]]:
446         """uid -> (прочитано, отвечено, маркер) для уже известных писем -
447         дешёвый способ подхватить чужие изменения флагов (другой клиент,
448         веб-интерфейс) без повторной загрузки заголовков."""
449         if not uids:
450             return {}
451         self._select(folder)
452         response = self._client.fetch(list(uids), ["FLAGS"])
453         result: dict[int, tuple[bool, bool, str | None]] = {}
454         for key, data in response.items():
455             uid = data.get(b"UID", key)
456             if uid is None:
457                 continue
458             flags = data.get(b"FLAGS", ())
459             marker = join_markers(_COLOR_BY_KEYWORD[f] for f in flags if f in _COLOR_BY_KEYWORD)
460             if marker is None and b"\\Flagged" in flags:
461                 marker = "red"
462             result[int(uid)] = (b"\\Seen" in flags, b"\\Answered" in flags, marker)
463         return result

```

```

465 @reconnecting
466 def search_uids(self, folder: str, *, before=None) -> list[int]:
467     """UID всех писем папки (или только тех, что старше даты `before`) –
468     в отличие от fetch_summaries/fetch_folder_summaries это не
469     ограничено последними `limit` письмами: нужно для массовой
470     выгрузки в архив (вся папка / всё до даты), где важна ПОЛНАЯ папка,
471     а не то, что сейчас показано в таблице."""
472     self._select(folder)
473     criteria = ["BEFORE", before.strftime("%d-%b-%Y")] if before else "ALL"
474     return list(self._client.search(criteria))

475
476 def fetch_message_content(self, folder: str, uid: int) -> MessageContent:
477     return extract_content(message_from_bytes(self.fetch_message_raw(folder, uid)))

478
479 @reconnecting
480 def fetch_message_raw(self, folder: str, uid: int) -> bytes:
481     """Полный RFC 822 письма как есть – нужен для выгрузки в архив без
482     потерь (в отличие от fetch_message_content, который уже разобрал бы
483     текст/вложения и потерял бы всё остальное, например точные заголовки)."""
484     self._select(folder)
485     response = self._client.fetch([uid], ["BODY.PEEK[ ]"])
486     return response[uid][b"BODY[ ]"]

487
488 @reconnecting
489 def set_read(self, folder: str, uid: int, read: bool) -> None:
490     self._select(folder)
491     if read:
492         self._client.add_flags([uid], [b"\\Seen"])
493     else:
494         self._client.remove_flags([uid], [b"\\Seen"])

495
496 @reconnecting
497 def set_answered(self, folder: str, uid: int) -> None:
498     """Ставит стандартный флаг \\Answered на письмо, на которое только
499     что был отправлен ответ – не снимается обратно (как и в других
500     почтовых клиентах, "ответили" необратимо для этого письма)."""
501     self._select(folder)
502     self._client.add_flags([uid], [b"\\Answered"])

503
504 @reconnecting
505 def set_marker(self, folder: str, uid: int, color: str | None, *, previous_color=UNKNOWN_MARKER) -> None:
506     """Ставит/снимает \\Flagged + наш цветной keyword-флаг.

507     Gmail принимает произвольные keyword-флаги без вопросов, но
508     реальный корпоративный сервер (обнаружено на VK Mail) отвечает
509     "BAD [PARSE] Unable to parse flag" на STORE с несколькими нашими
510     keyword'ами разом – по всей видимости, сервер не разрешает
511     произвольные (не объявленные в PERMANENTFLAGS) keyword-флаги
512     вообще. Поэтому: (1) снимаем/ставим keyword'ы по одному, а не
513     разом – один отклонённый не должен мешать остальным; (2) если
514     сервер в принципе не принимает цветной keyword, тихо откатываемся
515     на стандартный \\Flagged, чтобы разметка не ломалась полностью
516     из-за того, что сервер не умеет в цвета.

517     `previous_color` – если вызывающий код уже знает текущий маркер
518     (обычно да – он же его и показывает в таблице), снимаем ТОЛЬКО
519     этот один keyword вместо того, чтобы вслепую пытаться снять все
520     6 возможных цветов на каждую смену маркера. На реальном сервере с
521     заметной сетевой задержкой это была разница между 1-2 round trip'ами
522     и 6 (жалоба: "маркер на письмах устанавливается очень долго").
523     Если previous_color не передан – поведение как раньше (безопасный,
524     но медленный вариант "снять всё возможное")."""
525     if previous_color is not UNKNOWN_MARKER and previous_color == color:
526         return # уже в нужном состоянии – нечего менять, даже SELECT не нужен
527     self._select(folder)
528     all_keywords = list(MARKER_COLORS.values())
529     # `color`/`previous_color` – один цвет или несколько через запятую
530     # (см. split_markers): на письме может быть несколько маркеров.
531     new_keywords = [MARKER_COLORS[c] for c in split_markers(color) if c in MARKER_COLORS]
532     if previous_color is UNKNOWN_MARKER:
533         to_remove = list(all_keywords)
534     elif previous_color is None:
535         to_remove = [] # маркера не было – снимать нечего, кроме самого нового keyword'а ниже не нужно
536     else:
537         to_remove = [MARKER_COLORS[c] for c in split_markers(previous_color) if c in MARKER_COLORS]

538     if not new_keywords:
539         self._remove_flags_best_effort(uid, [b"\\Flagged", *to_remove])
540         return
541     self._remove_flags_best_effort(uid, [k for k in to_remove if k not in new_keywords])
542     try:
543         self._client.add_flags([uid], [b"\\Flagged", *new_keywords])
544     except IMAPClientError:
545         self._client.add_flags([uid], [b"\\Flagged"])

546
547 def _remove_flags_best_effort(self, uid: int, flags: list[bytes]) -> None:
548     for flag in flags:
549         try:
550             self._client.remove_flags([uid], [flag])

```



```

554         except IMAPClientError:
555             pass # флаг и так не поддерживается/не был установлен — не критично

557     @reconnecting
558     def move_messages(self, folder: str, uids: list[int], target_folder: str) -> None:
559         """Переносит письма в другую папку (например, в корзину) — атомарно,
560         если сервер поддерживает MOVE (RFC 6851), иначе COPY + удаление.

561         Если целевой папки ещё нет на сервере — создаёт её и повторяет
562         один раз, вместо того чтобы падать с сырым "[TRYCREATE] Folder
563         does not exist" (жалоба на правила сортировки почты: правило
564         может ссылаться на папку, которую пользователь только собирался
565         создать, а не создал заранее руками)."""
566         if not uids:
567             return
568         self._select(folder)
569         try:
570             self._move_or_copy(uids, target_folder)
571         except IMAPClientError as exc:
572             if "TRYCREATE" not in str(exc).upper():
573                 raise
574             self._client.create_folder(target_folder)
575             self._move_or_copy(uids, target_folder)

576     def _move_or_copy(self, uids: list[int], target_folder: str) -> None:
577         if self._client.has_capability("MOVE"):
578             self._client.move(uids, target_folder)
579         else:
580             self._client.copy(uids, target_folder)
581             self._client.delete_messages(uids)
582             self._client.expunge()

583     @reconnecting
584     def delete_messages(self, folder: str, uids: list[int]) -> None:
585         """Безвозвратное удаление (Shift+Удалить, либо удаление из самой корзины)."""
586         if not uids:
587             return
588         self._select(folder)
589         self._client.delete_messages(uids)
590         self._client.expunge()

591     def _select(self, folder: str) -> None:
592         # Папка уже открыта этой же сессией — второй SELECT только теряет время.
593         if self._selected_folder != folder:
594             self._client.select_folder(folder, readonly=False)
595             self._selected_folder = folder

602     def _iter_body_parts(message: Message):
603         """Как Message.walk(), но НЕ спускается внутрь message/rfc822
604         (пересланное письмо целиком как вложение) — Message.walk() у стандартной
605         библиотеки считает такую часть "multipart" (её единственный payload —
606         вложенный Message-объект) и лезет внутрь неё саму же, вместо того
607         чтобы отдать её как один цельный узел вызывающему коду."""
608         if message.get_content_type() == "message/rfc822":
609             yield message
610             return
611         if message.is_multipart():
612             for part in message.get_payload():
613                 yield from _iter_body_parts(part)
614         else:
615             yield message

618     def extract_content(message: Message) -> MessageContent:
619         # Заголовки живут на верхнем уровне сообщения независимо от того,
620         # multipart оно или нет — читаем их один раз, а не в каждой из веток
621         # ниже (жалоба: "при просмотре письма невидно его реквизитов"), и
622         # передаём во все ветки через один хелпер, а не повторяя 5 kwargs в
623         # каждом return.
624         header_kwargs = dict(
625             subject=_decode_header_text(message.get("Subject")),
626             from=_decode_header_text(message.get("From")),
627             to=_decode_header_text(message.get("To")),
628             cc=_decode_header_text(message.get("Cc")),
629             bcc=_decode_header_text(message.get("Bcc")),
630         )

631     if not message.is_multipart():
632         if message.get_content_type() == "text/plain":
633             # "or" — не просто "если текста вообще не было" (None), но и
634             # "если он декодировался в пустую строку" (например,
635             # get_payload(decode=True) не осилил конкретную кодировку и
636             # вернул None → _decode_payload даёт "") — иначе панель чтения
637             # оставалась молча пустой безо всякой подсказки, хотя реальный
638             # веб-интерфейс почты то же письмо показывал с текстом (жалоба:
639             # "некоторые письма открываются так [пусто], а на самом деле
640             # они такие [с текстом]").
641             return MessageContent(text=_decode_payload(message) or "(нет текстового содержимого)", **header_kwargs)

```

```

643     if message.get_content_type() == "text/html":
644         return MessageContent(
645             text="(письмо в формате HTML — предпросмотр текста недоступен)",
646             html=_decode_payload(message),
647             **header_kwargs,
648         )
649     if message.get_content_type() == "text/calendar":
650         return MessageContent(text="", attachments=[_calendar_attachment(message)], **header_kwargs)
651     return MessageContent(
652         text="(письмо в формате HTML — предпросмотр текста недоступен)", **header_kwargs
653     )

655     text: str | None = None
656     html: str | None = None
657     attachments: list[Attachment] = []
658     inline_images: dict[str, tuple[str, bytes]] = {}

660     for part in _iter_body_parts(message):
661         if part.get_content_type() == "message/rfc822":
662             # Пересланное письмо целиком как вложение (Outlook и другие
663             # клиенты часто пересылают именно так, без явного
664             # Content-Disposition: attachment на этой части) — раньше
665             # message.walk() спускался ВНУТРЬ него (Python считает
666             # message/rfc822 "multipart", раз её единственный payload —
667             # вложенный Message), и текст/HTML пересылаемого письма
668             # подмешивался в тело исходного вместо того, чтобы стать одним
669             # отдельным вложением — жалоба: "у письма есть вложение, но
670             # его нет в просмотре и при открытии".
671             nested = part.get_payload(0)
672             nested_subject = _decode_header_text(nested.get("Subject")) if nested else ""
673             filename = _decode_filename(part.get_filename()) or f"{nested_subject or 'Пересланное письмо'}.eml"
674             attachments.append(
675                 Attachment(
676                     filename=filename,
677                     content_type="message/rfc822",
678                     payload=nested.as_bytes() if nested else (part.get_payload(decode=True) or b""),
679                 )
680             )
681             continue

683             filename = _decode_filename(part.get_filename())
684             content_type = part.get_content_type()
685             content_disposition = part.get_content_disposition() # 'attachment' | 'inline' | None
686             content_id = (part.get("Content-Id") or "").strip().strip("<>")

688             # Картинка со своим Content-Id — то, на что ссылается  в HTML-теле, а не отдельное вложение для скачивания
690             # (даже если у неё есть имя файла и/или Content-Disposition).
691             if content_id and content_type.startswith("image/"):
692                 inline_images[content_id] = (content_type, part.get_payload(decode=True) or b "")
693                 continue

695             # text/plain и text/html — кандидаты в само тело письма, а не во
696             # вложение, даже если у части задан Content-Type: ...; name="..."
697             # (part.get_filename() читает и его, не только
698             # Content-Disposition: filename=) — реальные HTML-рассылки
699             # (например, от Авито) так подписывают HTML-часть письма без
700             # всякого намерения сделать её вложением. Раньше bool(filename)
701             # ниже срабатывал именно на этом и уводил всё письмо во вложение,
702             # оставляя тело пустым (жалоба: "письма в формате html не
703             # просматриваются"). Вложением текстовая часть считается только
704             # при явном Content-Disposition: attachment.
705             is_body_part = content_type in ("text/plain", "text/html") and content_disposition != "attachment"

707             # text/calendar (RFC 5546 iTPP-приглашение) сохраняем как вложение
708             # всегда — не только когда отправитель явно проставил
709             # Content-Disposition: attachment/filename (не все серверы это
710             # делают), иначе приглашение молча потеряется.
711             is_attachment = not is_body_part and (
712                 bool(filename) or content_disposition == "attachment" or content_type == "text/calendar"
713             )
714             if is_attachment:
715                 attachments.append(
716                     _calendar_attachment(part) if content_type == "text/calendar" else Attachment(
717                         filename=filename or "(без имени)",
718                         content_type=content_type,
719                         payload=part.get_payload(decode=True) or b "",
720                     )
721                 )
722             continue

724             if content_type == "text/plain" and text is None:
725                 text = _decode_payload(part)
726             elif content_type == "text/html" and html is None:
727                 html = _decode_payload(part)

729     if not text:
730         # "not text", а не "text is None" — часть text/plain могла найтись,
731         # но не осилить декодирование (get_payload(decode=True) вернул

```

```

732     # None → _decode_payload дала "" и оставить панель чтения молча
733     # пустой без единой подсказки (жалоба: "некоторые письма
734     # открываются так [пусто], а на самом деле они такие [с текстом]").
735     text = "(письмо в формате HTML — предпросмотр текста недоступен)" if html else "(нет текстового
содержимого)"

737     return MessageContent(
738         text=text,
739         attachments=attachments,
740         html=html or "",
741         inline_images=inline_images,
742         **header_kwargs,
743     )

746 def _calendar_attachment(part: Message) -> Attachment:
747     return Attachment(
748         filename=_decode_filename(part.get_filename()) or "invite.ics",
749         content_type="text/calendar",
750         payload=part.get_payload(decode=True) or b"",
751     )

754 def _decode_header_text(value: str | None) -> str:
755     """Как _decode_filename, но для обычных текстовых заголовков (Subject/
756     From/To/Cc) — не все сервера сворачивают RFC 2047 encoded-word до
757     ENVELOPE, который парсит imapclient (см. _decode_subject); здесь тот
758     же случай, но для содержимого письма, разбираемого через email.message."""
759     if isinstance(value, Header):
760         # Заголовок с «сырыми» не-ASCII байтами (RFC 6532 / небрежные
761         # отправители): email.message отдаёт Header с charset unknown-8bit,
762         # внутри — строка с surrogateescape. Восстанавливаем байты и
763         # читаем как UTF-8; иначе падало на проверке "=?" in value
764         # (TypeError: Header не итерируется).
765         parts: list[str] = []
766         for chunk, charset in decode_header(value):
767             parts.append(decode_bytes_safely(chunk, charset) if isinstance(chunk, bytes) else chunk)
768         value = "".join(parts)
769     if not value or "=? " not in value:
770         return value or ""
771     return _decode_rfc2047(value.encode("ascii", errors="replace"))

774 def _decode_filename(filename: str | None) -> str | None:
775     """get_filename() отдаёт значение как есть — некоторые сервера (в т.ч.
776     встречались от Exchange) кодируют имя файла в RFC 2047 (=?utf-8?B?...?)
777     вместо RFC 2231, которое email.message понимает само. Без декодирования
778     имя вложения показывалось бы пользователю нечитаемой кодированной
779     строкой вместо настоящего имени файла."""
780     if not filename or "=? " not in filename:
781         return filename
782     return _decode_rfc2047(filename.encode("ascii", errors="replace"))

785 def _decode_payload(part: Message) -> str:
786     payload = part.get_payload(decode=True) or b""
787     charset = part.get_content_charset() or "utf-8"
788     return payload.decode(charset, errors="replace")

791 def _to_summary(data: dict) -> MessageSummary:
792     envelope = data[b"ENVELOPE"]
793     sender_display, sender_email = _format_address(envelope.from_)
794     message_id = envelope.message_id
795     flags = data.get(b"FLAGS", ())
796     marker_color = join_markers(_COLOR_BY_KEYWORD[f] for f in flags if f in _COLOR_BY_KEYWORD)
797     if marker_color is None and b"\\Flagged" in flags:
798         # Жалоба: "не сохраняется проставленный маркер, через какое-то
799         # время пропадает" — сервер (VK Mail) не хранит произвольные
800         # keyword-флаги ($RedMailRed и т.п.: их нет в PERMANENTFLAGS),
801         # принимает их только на время сессии и молча теряет, а стандартный
802         # \\Flagged хранит. Раньше без keyword'a маркер считался снятым —
803         # теперь \\Flagged без цвета = маркер по умолчанию; конкретный цвет
804         # при этом восстанавливается из локального кэша (см.
805         # CachedMailbox.refresh_folder).
806         marker_color = "red"
807     return MessageSummary(
808         uid=data[b"UID"],
809         subject=_decode_subject(envelope.subject),
810         sender=sender_display,
811         sender_email=sender_email,
812         date=envelope.date.strftime("%Y-%m-%d %H:%M") if envelope.date else "",
813         message_id=message_id.decode("ascii", errors="replace") if message_id else "",
814         has_attachments=_body_has_attachment(data[b"BODYSTRUCTURE"]) if b"BODYSTRUCTURE" in data else False,
815         marker_color=marker_color,
816         importance=parse_importance(message_from_bytes(data.get(b"BODY[HEADER.FIELDS (IMPORTANCE X-PRIORITY)]"),
b""))),
817         is_read=b"\\Seen" in flags,
818         to=_format_address_list(getattr(envelope, "to", None)),

```

```

819         is_answered=b"\\Answered" in flags,
820         size=int(data.get(b"RFC822.SIZE", 0) or 0),
821     )

824 def _body_has_attachment(structure) -> bool:
825     """Смотрит в BODYSTRUCTURE, не скачивая тело письма целиком.

827     BODYSTRUCTURE – сырая вложенная структура по RFC 3501 (см. imapclient
828     response_types.BodyData), без готового поля "это вложение". Ищем
829     Content-Disposition: attachment паттерн-мэтчингом (2-элементный кортеж
830     вида (b'ATTACHMENT', (...))), а не по фиксированному индексу – точная
831     позиция disposition в кортеже "плавает" в зависимости от типа part'a
832     (у text/* и message/rfc822 есть дополнительные поля перед ней).
833     """
834     if structure.is_multipart:
835         parts, rest = structure[0], structure[1:]
836         if _disposition_is_attachment(rest):
837             return True
838         return any(_body_has_attachment(part) for part in parts)
839     # message/rfc822 (пересланное письмо целиком) – общепринято вложение
840     # само по себе, независимо от явного Content-Disposition: attachment
841     # (многие клиенты, включая Outlook, его не выставляют на такой части) –
842     # без этого скрепка в списке писем не показывалась вовсе (жалоба:
843     # "у письма есть вложение, но его нет в просмотре и при открытии").
844     if len(structure) >= 2 and _field_upper(structure[0]) == "MESSAGE" and _field_upper(structure[1]) == "RFC822":
845         return True
846     return _disposition_is_attachment(structure)

849 def _field_upper(value) -> str:
850     if isinstance(value, bytes):
851         return value.decode("ascii", errors="replace").upper()
852     return str(value).upper()

855 def _disposition_is_attachment(fields) -> bool:
856     for value in fields:
857         if isinstance(value, (tuple, list)) and len(value) == 2 and isinstance(value[0], bytes):
858             if value[0].upper() == b"ATTACHMENT":
859                 return True
860     return False

863 def parse_importance(headers: Message) -> str:
864     """Классифицирует важность письма по заголовкам Importance/X-Priority.

866     Публичная – используется и для IMAP (заголовки приходят отдельным полем
867     FETCH), и для archive_store (заголовки уже есть в разобранном письме)."""
868     importance = (headers.get("Importance") or "").strip().lower()
869     if importance in ("high", "urgent"):
870         return "high"
871     if importance == "low":
872         return "low"
873     priority = (headers.get("X-Priority") or "").strip()
874     if priority[:1] in ("1", "2"):
875         return "high"
876     if priority[:1] in ("4", "5"):
877         return "low"
878     return "normal"

881 def _decode_subject(raw: bytes | None) -> str:
882     if not raw:
883         return "(без темы)"
884     return _decode_rfc2047(raw)

887 def _decode_rfc2047(raw: bytes) -> str:
888     # Имена отправителей и темы писем сервер отдаёт как есть – они бывают
889     # в кодированных словах RFC 2047 (=?utf-8?B?...?), а не только сырым UTF-8.
890     if b"=?" not in raw:
891         # Нет encoded-word – это либо чистый ASCII, либо сырой UTF-8
892         # (RFC 6532; реальная находка на Dovecot: тема превращалась в
893         # знаки замены, потому что байты читались как ASCII).
894         return raw.decode("utf-8", errors="replace")
895     # latin-1 – обратимое 1:1 отображение байтов в символы: незакодированные
896     # куски decode_header вернёт как те же байты (raw-unicode-escape для
897     # символов < 256), и их можно прочитать как UTF-8.
898     parts = decode_header(raw.decode("latin-1"))
899     out: list[str] = []
900     for chunk, encoding in parts:
901         if isinstance(chunk, bytes):
902             out.append(decode_bytes_safely(chunk, encoding))
903         else:
904             out.append(chunk.encode("latin-1", errors="replace").decode("utf-8", errors="replace"))
905     return "".join(out)

```

```

908 def decode_bytes_safely(chunk: bytes, encoding: str | None) -> str:
909     """Байты заголовка → текст: неизвестная/служебная кодировка (например,
910     'unknown-8bit', которую подставляет email.header для сырых 8-битных
911     заголовков; реальная находка на автоархиве Gmail: "unknown encoding:
912     unknown-8bit") читается как UTF-8, а не роняет разбор письма."""
913     name = (encoding or "utf-8").strip().lower()
914     if name in ("", "unknown-8bit", "x-unknown", "unknown", "default", "8bit", "binary"):
915         name = "utf-8"
916     try:
917         return chunk.decode(name, errors="replace")
918     except LookupError:
919         return chunk.decode("utf-8", errors="replace")

922 def _format_address_list(addresses) -> str:
923     """Все адреса списка (To/Cc), через запятую — используется только для
924     отображения в списке писем, не для машинного разбора (см.
925     _parse_recipient_list в main_window.py для этого)."""
926     if not addresses:
927         return ""
928     parts = []
929     for address in addresses:
930         mailbox = address.mailbox.decode("utf-8", errors="replace") if address.mailbox else ""
931         host = address.host.decode("utf-8", errors="replace") if address.host else ""
932         email = f"{mailbox}@{host}" if mailbox and host else mailbox
933         name = _decode_rfc2047(address.name) if address.name else ""
934         parts.append(f"{name} <{email}>" if name and email else (email or name))
935     return ", ".join(parts)

938 def _format_address(addresses) -> tuple[str, str]:
939     """Возвращает (отображаемое имя, email-адрес) первого адреса в списке."""
940     if not addresses:
941         return "(неизвестно)", ""
942     address = addresses[0]
943     mailbox = address.mailbox.decode("utf-8", errors="replace") if address.mailbox else ""
944     host = address.host.decode("utf-8", errors="replace") if address.host else ""
945     email = f"{mailbox}@{host}" if mailbox and host else mailbox
946     if address.name:
947         return _decode_rfc2047(address.name), email
948     return (email or "(неизвестно)", email)

==== src/redmail/mailbox.py (223 строк) ====
1  from __future__ import annotations

3  import threading
4  from pathlib import Path

6  from pathlib import Path as _Path

8  from redmail import archive_store, cache_store, sync_engine
9  from redmail.applog import get_logger
10 from redmail.imap_client import UNKNOWN_MARKER, Account, ImapSession, MessageContent, MessageSummary

12 _log = get_logger("mailbox")

15 class CachedMailbox:
16     """Ящик поверх локальной копии (аналог OST): интерфейс читает только
17     из базы, сервер опрашивается синхронизацией.

19     folder_summaries() — чистое чтение из базы, сети не касается.
20     refresh_folder() — синхронизация заголовков папки с сервером
21     (sync_engine.sync_folder_headers): новые письма добавляются, удалённые
22     на сервере удаляются локально, флаги обновляются; тела при этом не
23     скачиваются. Первое открытие ещё не синхронизированной папки тоже
24     идёт через refresh_folder.

26     message_content() — тело из базы; если его ещё нет (фон не успел или
27     письмо большое и отложено) — скачивается сейчас и сохраняется.

29     sync_all()/download_bodies() — полная фоновая синхронизация (все папки,
30     затем тела от новых к старым), см. sync_engine.
31     """

33     def __init__(self, session: ImapSession, account: Account, *, body_max_bytes: int =
sync_engine.DEFAULT_BODY_MAX_BYTES):
34         self.session = session
35         self._account_key = f"{account.host}:{account.username}"
36         self.body_max_bytes = body_max_bytes
37         # Одна синхронизация за раз на ящик: обновление по кнопке/таймеру и
38         # полный фоновый проход могли стартовать одновременно и оба качать
39         # одни и те же заголовки (в журнале на реальном ящике: две строки
40         # «новых 3096» подряд). Второй ждёт первого и находит папку уже
41         # синхронизированной.
42         self._sync_lock = threading.Lock()

44     @property
45     def account_key(self) -> str:

```

```

46         return self._account_key

48     def folder_summaries(self, folder: str, limit: int | None = None) -> list[MessageSummary]:
49         # Только база, никогда сеть: вызывается из потока интерфейса, а
50         # синхронизация может в этот момент держать замок ящика – ждать её
51         # здесь значило бы заморозить окно. Не синхронизированную папку
52         # интерфейс запрашивает через refresh_folder в фоне (см.
53         # is_folder_synced).
54         return cache_store.get_folder_summaries(self._account_key, folder, limit)

56     def is_folder_synced(self, folder: str) -> bool:
57         return cache_store.is_folder_synced(self._account_key, folder)

59     def folder_message_total(self, folder: str) -> int:
60         return cache_store.count_folder_summaries(self._account_key, folder)

62     def refresh_folder(self, folder: str, limit: int | None = None, *, progress=None, stop: threading.Event | None = None) -> list[MessageSummary]:
63         with self._sync_lock:
64             sync_engine.sync_folder_headers(self.session, self._account_key, folder, progress=progress, stop=stop)
65             return cache_store.get_folder_summaries(self._account_key, folder, limit)

67     def sync_all(self, folders: list[str], *, progress=None, stop: threading.Event | None = None) -> sync_engine.SyncStats:
68         with self._sync_lock:
69             return sync_engine.sync_all_folders(self.session, self._account_key, folders, progress=progress, stop=stop)

71     # Папки, тела которых фоном не качаются (зеркало всех писем у Gmail).
72     skip_body_folders: tuple[str, ...] = ()

74     def pending_bodies(self) -> int:
75         return cache_store.count_messages_without_body(self._account_key, self.body_max_bytes, skip_folders=self.skip_body_folders)

77     def download_bodies(self, *, progress=None, stop: threading.Event | None = None, limit: int | None = None) -> int:
78         with self._sync_lock:
79             return sync_engine.download_bodies(
80                 self.session, self._account_key, max_bytes=self.body_max_bytes, progress=progress, stop=stop, limit=limit,
81                 skip_folders=self.skip_body_folders,
82             )

84     def message_content(self, folder: str, uid: int) -> MessageContent:
85         cached = cache_store.get_message_content(self._account_key, folder, uid)
86         if cached is not None:
87             return cached
88         ref = cache_store.get_archive_ref(self._account_key, folder, uid)
89         if ref is not None:
90             # Единый индекс: письмо перенесено автоархивом – тело из файла архива.
91             return archive_store.get_message_content(_Path(ref[0]), ref[1])
92         content = self.session.fetch_message_content(folder, uid)
93         cache_store.save_message_content(self._account_key, folder, uid, content)
94         return content

96     def message_raw(self, folder: str, uid: int) -> bytes:
97         """Не кэшируется – нужен только для разового экспорта в архив."""
98         return self.session.fetch_message_raw(folder, uid)

100     def search_uids(self, folder: str, *, before=None) -> list[int]:
101         """Не кэшируется – используется только для массовой выгрузки папки
102         в архив (вся папка / всё до даты), где важна полная папка на
103         сервере, а не то, что сейчас в локальном кэше сводок."""
104         return self.session.search_uids(folder, before=before)

106     def _archived(self, folder: str, uid: int) -> tuple[str, int] | None:
107         return cache_store.get_archive_ref(self._account_key, folder, uid)

109     def set_marker(self, folder: str, uid: int, color: str | None, *, previous_color=UNKNOWN_MARKER) -> None:
110         ref = self._archived(folder, uid)
111         if ref is not None:
112             archive_store.set_marker(_Path(ref[0]), ref[1], color) # на сервере письма уже нет
113         else:
114             self.session.set_marker(folder, uid, color, previous_color=previous_color)
115         cache_store.set_marker(self._account_key, folder, uid, color)

117     def set_read(self, folder: str, uid: int, read: bool) -> None:
118         if self._archived(folder, uid) is None:
119             self.session.set_read(folder, uid, read)
120         cache_store.set_read(self._account_key, folder, uid, read)

122     def set_answered(self, folder: str, uid: int) -> None:
123         if self._archived(folder, uid) is None:
124             self.session.set_answered(folder, uid)
125         cache_store.set_answered(self._account_key, folder, uid)

127     def delete_on_server(self, folder: str, uids: list[int]) -> None:
128         """Только сервер, без локальной базы – для автоархива, который

```

```

129         оставляет строку в индексе."""
130         self.session.delete_messages(folder, uids)

132     def move_to_trash(self, folder: str, uids: list[int], trash_folder: str) -> None:
133         self.move_to_folder(folder, uids, trash_folder)

135     def _split_archived(self, folder: str, uids: list[int]) -> tuple[list[int], list[tuple[int, str, int]]]:
136         live: list[int] = []
137         archived: list[tuple[int, str, int]] = []
138         for uid in uids:
139             ref = self._archived(folder, uid)
140             if ref is None:
141                 live.append(uid)
142             else:
143                 archived.append((uid, ref[0], ref[1]))
144         return live, archived

146     def move_to_folder(self, folder: str, uids: list[int], target_folder: str) -> None:
147         """Общий переезд писем в любую папку (не только корзину) – так же
148         используется при ручном применении правил сортировки почты.
149         Архивные письма (их уже нет на сервере) при «переезде в корзину»
150         удаляются из архива и индекса."""
151         live, archived = self._split_archived(folder, uids)
152         if live:
153             self.session.move_messages(folder, live, target_folder)
154             cache_store.delete_messages(self._account_key, folder, live)
155         if archived:
156             self._delete_archived(folder, archived)

158     def _delete_archived(self, folder: str, archived: list[tuple[int, str, int]]) -> None:
159         by_file: dict[str, list[int]] = {}
160         for _uid, path, archive_uid in archived:
161             by_file.setdefault(path, []).append(archive_uid)
162         for path, ids in by_file.items():
163             try:
164                 archive_store.delete_messages(_Path(path), ids)
165             except Exception as exc:
166                 _log.warning("Архив %s: не удалось удалить письма %s: %s", path, ids, exc)
167         cache_store.delete_messages(self._account_key, folder, [uid for uid, _p, _a in archived])

169     def delete_messages(self, folder: str, uids: list[int]) -> None:
170         # Удаление локально = удаление на сервере (договорённость по
171         # хранилищу): сначала сервер, затем локальная копия.
172         live, archived = self._split_archived(folder, uids)
173         if live:
174             self.session.delete_messages(folder, live)
175             cache_store.delete_messages(self._account_key, folder, live)
176         if archived:
177             self._delete_archived(folder, archived)

179     def close(self) -> None:
180         self.session.close()

182     def append_message(self, folder: str, raw: bytes, *, flags: tuple = ()) -> None:
183         # Новое письмо подхватит следующая синхронизация папки (появится
184         # новый UID на сервере).
185         self.session.append_message(folder, raw, flags=flags)

188 class ArchiveSource:
189     """Тот же протокол, что у CachedMailbox (folder_summaries/refresh_folder/
190     message_content/set_marker/delete_messages/close), но поверх локального
191     файла архива – чтобы UI не знал разницы между «живой ящик» и «архив»."""

193     def __init__(self, path: Path):
194         self.path = path

196     def folder_summaries(self, folder: str, limit: int | None = None) -> list[MessageSummary]:
197         messages = archive_store.list_messages(self.path, folder)
198         return messages[:limit] if limit is not None else messages

200     def folder_message_total(self, folder: str) -> int:
201         return len(archive_store.list_messages(self.path, folder))

203     def refresh_folder(self, folder: str, limit: int | None = None, **kwargs) -> list[MessageSummary]:
204         # Архив не меняется извне сам по себе – «обновить» просто перечитывает файл.
205         return self.folder_summaries(folder, limit)

207     def message_content(self, folder: str, uid: int) -> MessageContent:
208         return archive_store.get_message_content(self.path, uid)

210     def set_marker(self, folder: str, uid: int, color: str | None, *, previous_color=UNKNOWN_MARKER) -> None:
211         archive_store.set_marker(self.path, uid, color)

213     def set_read(self, folder: str, uid: int, read: bool) -> None:
214         pass # архив всегда "прочитан" (см. folder_summaries) – переключать нечего

216     def set_answered(self, folder: str, uid: int) -> None:
217         pass # архив – статичный снимок, отвечать "заново" на его письма нельзя

```

```

219     def delete_messages(self, folder: str, uids: list[int]) -> None:
220         archive_store.delete_messages(self.path, uids)

222     def close(self) -> None:
223         pass

==== src/redmail/smtp_client.py (196 строк) ====
1  from __future__ import annotations

3  import smtplib
4  from collections.abc import Callable
5  from dataclasses import dataclass, field
6  from email import policy
7  from email.message import EmailMessage
8  from email.utils import formatdate, getaddresses, make_msgid

10 from redmail.applog import get_logger

12 _log = get_logger("smtp")

14 # Жалоба: "ошибка отправки приходит именно если отправить из redmail, а
15 # при отправке из VK всё уходит без ошибок" — реальная причина не в
16 # запятой в поле "Кому" (тот путь уже фильтрует пустые адреса, см.
17 # _parse_recipient_list в main_window.py), а в том, что политика по
18 # умолчанию для EmailMessage — email.policy.default с cte_type="8bit":
19 # для любого текста с кириллицей (почти каждое письмо здесь) это даёт
20 # Content-Transfer-Encoding: 8bit. Но smtplib.SMTP.send_message() решает,
21 # слать ли ESMTP-параметр BODY=8BITMIME, ТОЛЬКО по тому, содержат ли
22 # non-ASCII символы САМИ АДРЕСА конверта (envelope from/to) — адреса это
23 # обычные email на латинице, поэтому BODY=8BITMIME не запрашивается,
24 # и письмо с 8-битным телом уходит без него: формальное нарушение RFC
25 # 6152, которое строгие корпоративные шлюзы контентной фильтрации вполне
26 # резонно отклоняют уже после приёма (SMTP error ... after end of data:
27 # 500 Message rejected) — то самое поведение, "уходит, но потом
28 # отбойник". cte_type="7bit" заставляет content manager всегда кодировать
29 # нелатинский текст через quoted-printable/base64 (7-битно чистые,
30 # универсально совместимые кодировки) вместо сырых 8-битных байт —
31 # независимо от того, что там на стороне сервера с 8BITMIME.
32 _OUTGOING_POLICY = policy.default.clone(cte_type="7bit")

35 @dataclass
36 class SmtAccount:
37     host: str
38     username: str
39     password: str
40     port: int = 587
41     use_ssl: bool = False # True = неявный TLS (порт 465), False = STARTTLS (порт 587)
42     # "password" — обычный AUTH LOGIN/PLAIN; "kerberos" — SSO для сервера в
43     # домене, см. Account.auth_type в imap_client.py и gssapi_sasl.py.
44     auth_type: str = "password"
45     # См. Account.keytab_path/principal — тот же необязательный keytab.
46     keytab_path: str = ""
47     principal: str = ""

50 @dataclass
51 class OutgoingAttachment:
52     filename: str
53     content_type: str
54     payload: bytes
55     # Доп. параметры Content-Type — нужно для приглашений: MIME-параметр
56     # method= (RFC 5546) рядом с METHOD: внутри самого .ics — многие клиенты
57     # (Outlook в их числе) ищут именно внешний параметр, чтобы показать
58     # интерфейс "принять/отклонить", а не просто вложение.
59     content_type_params: dict[str, str] = field(default_factory=dict)

62 @dataclass
63 class OutgoingMessage:
64     sender: str
65     to: list[str]
66     subject: str
67     body: str
68     cc: list[str] = field(default_factory=list)
69     bcc: list[str] = field(default_factory=list)
70     in_reply_to: str | None = None
71     references: list[str] = field(default_factory=list)
72     attachments: list[OutgoingAttachment] = field(default_factory=list)
73     # Отредактированное форматирование/встроенные картинки из ComposeDialog
74     # (жалоба: "нет возможности вставить картинку... не даёт установить
75     # какие-либо шрифты"). None — письмо чисто текстовое, шлём как раньше;
76     # когда задано, `body` остаётся обычным текстовым fallback-содержимым
77     # (RFC 2046 multipart/alternative — получатели без поддержки HTML
78     # видят его), а html_body — реальное форматированное содержимое.
79     html_body: str | None = None
80     # cid -> (content_type, payload) — картинки, вставленные прямо в текст
81     # письма (не файловые вложения), связываются с html_body через

```



```

82     #  так же, как их парсит imap_client.extract_content
83     # у входящей почты.
84     inline_images: dict[str, tuple[str, bytes]] = field(default_factory=dict)

87 def build_email_message(message: OutgoingMessage) -> EmailMessage:
88     """Общая сборка RFC 822 письма — используется и для реальной отправки
89     по SMTP, и для того, чтобы положить готовую копию письма прямо в
90     "Отправленные"/"Черновики" через IMAP APPEND (сервер не всегда сам
91     сохраняет копию исходящих — жалоба: "не отображается отправка почты,
92     не появляется в папке отправленные")."""
93     email_message = EmailMessage(policy=OUTGOING_POLICY)
94     email_message["From"] = message.sender
95     email_message["To"] = ", ".join(message.to)
96     if message.cc:
97         email_message["Cc"] = ", ".join(message.cc)
98     if message.bcc:
99         # smtplib.send_message() сам добавляет адреса из Bcc в список
100        # получателей SMTP-конверта и одновременно вырезает сам заголовок
101        # Bcc из фактически передаваемого письма (см. исходники smtplib) —
102        # получателям Bcc не виден друг друга и остальным получателям.
103        email_message["Bcc"] = ", ".join(message.bcc)
104    email_message["Subject"] = message.subject
105    email_message["Date"] = formatdate(localtime=True)
106    email_message["Message-Id"] = make_msgid()
107    if message.in_reply_to:
108        email_message["In-Reply-To"] = message.in_reply_to
109        email_message["References"] = " ".join([*message.references, message.in_reply_to])
110    email_message.set_content(message.body)
111    if message.html_body:
112        email_message.add_alternative(message.html_body, subtype="html")
113        if message.inline_images:
114            html_part = email_message.get_payload()[-1]
115            for cid, (content_type, payload) in message.inline_images.items():
116                maintype, _, subtype = content_type.partition("/")
117                html_part.add_related(payload, maintype=maintype or "image", subtype=subtype or "png",
cid=f"<cid:{cid}>")

119    for attachment in message.attachments:
120        maintype, _, subtype = attachment.content_type.partition("/")
121        email_message.add_attachment(
122            attachment.payload,
123            maintype=maintype or "application",
124            subtype=subtype or "octet-stream",
125            filename=attachment.filename,
126            params=attachment.content_type_params or None,
127        )
128    return email_message

131 def _with_retry(operation: Callable[[], None]) -> None:
132     """Один повтор на свежем соединении при обрыве на любом этапе —
133     коннект, STARTTLS, аутентификация (в т.ч. GSSAPI) или сама отправка.
134     Раньше единственный обрыв TCP/TLS падал прямо в интерфейс сырым
135     "Connection unexpectedly closed" (smtplib.SMTPServerDisconnected,
136     который наследуется от OSError) — жалоба при отправке приглашения на
137     встречу: "Встреча сохранена, но не разослана". Тот же принцип, что
138     уже применён для IMAP в imap_client.py._reconnecting."""
139     try:
140         operation()
141     except (OSError, EOFError) as exc:
142         _log.warning("SMTP: обрыв соединения (%s), повтор на новом соединении", exc)
143         operation()

146 def _connect_and_authenticate(account: SmtplibAccount) -> smtplib.SMTP:
147     smtp_cls = smtplib.SMTP_SSL if account.use_ssl else smtplib.SMTP
148     client = smtp_cls(account.host, account.port, timeout=30)
149     if not account.use_ssl:
150         client.starttls()
151     if account.auth_type == "kerberos":
152         # Импорт внутри функции — см. imap_client.py._login: gssapi
153         # нужен только для SSO и не должен ломать обычный пароль там,
154         # где нет системных библиотек Kerberos.
155         from redmail import gssapi_sasl

157         gssapi_sasl.smtp_sasl_login(
158             client,
159             account.host,
160             account.username,
161             keytab_path=account.keytab_path,
162             principal=account.principal,
163         )
164     else:
165         client.login(account.username, account.password)
166     _log.info("SMTP %s:%s: вход выполнен (%s, %s)", account.host, account.port, account.username,
account.auth_type)
167     return client

```

```

170 def test_connection(account: SmtAccount) -> None:
171     """Подключается и проходит аутентификацию, ничего не отправляя — для
172     кнопки "Проверить подключение" в настройках (жалоба-пожелание:
173     "может добавить кнопку проверки подключения для входящих, исходящих
174     и календаря?"). Успех — соединение установлено и закрыто без ошибок;
175     любая проблема (сеть, TLS, логин/SSO) всплывает как исключение."""
176     def attempt() -> None:
177         with _connect_and_authenticate(account):
178             pass
179
180     _with_retry(attempt)

183 def send_message(account: SmtAccount, message: OutgoingMessage) -> None:
184     email_message = build_email_message(message)

186     def attempt() -> None:
187         with _connect_and_authenticate(account) as client:
188             client.send_message(email_message)

190     recipients = getaddresses(email_message.get_all("To", []) + email_message.get_all("Cc", []) +
email_message.get_all("Bcc", []))
191     try:
192         _with_retry(attempt)
193     except Exception as exc:
194         _log.error("SMTP %s: отправка не удалась (получателей %d, тема %r): %s", account.host, len(recipients),
message.subject, exc)
195         raise
196     _log.info("SMTP %s: письмо отправлено (получателей %d, тема %r)", account.host, len(recipients),
message.subject)

==== src/redmail/itip.py (301 строк) ====
1 from __future__ import annotations

3 import base64
4 from dataclasses import dataclass, replace
5 from datetime import date, datetime, timezone
6 from email.message import Message
7 from pathlib import Path

9 import icalendar

11 from redmail import calendar_store
12 from redmail.calendar_store import Attendee, Event, new_uid
13 from redmail.imap_client import Attachment

15 # iTIP (RFC 5546) поверх обычной почты — единственный способ приглашений/
16 # ответов/переносов, который одинаково работает через Exchange, VK Mail и
17 # любой другой сервер по IMAP/SMTP: сам .ics-формат встраивания в письмо
18 # (text/calendar, METHOD:REQUEST/REPLY/CANCEL) — общий стандарт, которому
19 # следует и Outlook, а не что-то специфичное для одного сервера.

21 _PARTSTAT_TO_LOCAL = {
22     "ACCEPTED": "accepted",
23     "DECLINED": "declined",
24     "TENTATIVE": "tentative",
25     "NEEDS-ACTION": "needs-action",
26 }
27 _PARTSTAT_TO_ICAL = {v: k for k, v in _PARTSTAT_TO_LOCAL.items()}

30 @dataclass
31 class IncomingInvite:
32     method: str # "REQUEST" | "REPLY" | "CANCEL" | ...
33     event: Event
34     replying_attendee_email: str = "" # заполнено только для method == "REPLY"

37 class NotAnInviteError(Exception):
38     """В письме нет разбираемого text/calendar с VEVENT."""

41 def find_calendar_part(message: Message) -> bytes | None:
42     """Ищет вложенный .ics (text/calendar — так его встраивает любой iTIP-
43     совместимый отправитель: Outlook/Exchange, VK Mail, сам redmail) в уже
44     разобранном письме."""
45     if message.get_content_type() == "text/calendar":
46         return message.get_payload(decode=True)
47     if message.is_multipart():
48         for part in message.walk():
49             if part.get_content_type() == "text/calendar":
50                 return part.get_payload(decode=True)
51     return None

54 def parse_invite(ics_bytes: bytes, my_email: str) -> IncomingInvite:
55     cal = icalendar.Calendar.from_ical(ics_bytes)
56     method = str(cal.get("method", "REQUEST")).upper()

```

```

58     vevent = None
59     for component in cal.walk():
60         if component.name == "VEVENT":
61             vevent = component
62             break
63     if vevent is None:
64         raise NotAnInviteError("B .ics нет VEVENT")

66     event = _event_from_vevent(vevent, method, my_email, ics_bytes)
67     replying_attendee_email = event.attendees[0].email if method == "REPLY" and event.attendees else ""
68     return IncomingInvite(method=method, event=event, replying_attendee_email=replying_attendee_email)

71 def parse_ics_events(ics_bytes: bytes, my_email: str) -> list[Event]:
72     """Разбирает ВСЕ VEVENT из .ics — экспорт целого календаря (несколько
73     встреч без METHOD:, в отличие от одиночного приглашения), например
74     файл, выгруженный из VK Mail/Google/Outlook через "Экспорт календаря".
75     Записи, которые не удалось разобрать (битые/неполные), пропускаются —
76     одна плохая запись не должна срывать импорт всего файла."""
77     cal = icalendar.Calendar.from_ical(ics_bytes)
78     method = str(cal.get("method", "PUBLISH")).upper()
79     events: list[Event] = []
80     for component in cal.walk():
81         if component.name != "VEVENT":
82             continue
83         try:
84             events.append(_event_from_vevent(component, method, my_email, None))
85         except (KeyError, ValueError):
86             continue
87     return events

90 def import_ics(path: Path, ics_bytes: bytes, my_email: str = "", calendar_id: str | None = None) -> int:
91     """Импортирует все события из .ics-файла (выгрузка целого календаря
92     внешней системой — например, "Экспорт" из VK Mail/Google/Outlook, а
93     не одиночное приглашение) в локальный calendar_store.

94     calendar_id — куда положить импортированные события. Раньше не
95     передавался вовсе, и события всегда попадали в календарь по умолчанию,
96     смешиваясь с личными встречами — не было способа получить именно
97     "календарь из другого источника" отдельным списком со своим
98     цветом/видимостью (жалоба: "не создаётся календарь из другого
99     источника"). None — прежнее поведение (calendar_store.DEFAULT_CALENDAR_ID).
100     """
101     events = parse_ics_events(ics_bytes, my_email)
102     if calendar_id is not None:
103         events = [replace(event, calendar_id=calendar_id) for event in events]
104     for event in events:
105         calendar_store.save_event(path, event)
106     return len(events)

110 def _event_from_vevent(vevent, method: str, my_email: str, raw_ics: bytes | None) -> Event:
111     dtstart_value = vevent["DTSTART"].dt
112     all_day = not isinstance(dtstart_value, datetime) and isinstance(dtstart_value, date)
113     dtstart = _to_utc(dtstart_value)
114     dtend_prop = vevent.get("DTEND")
115     dtend = _to_utc(dtend_prop.dt) if dtend_prop is not None else dtstart

117     organizer_prop = vevent.get("ORGANIZER")
118     organizer_email = _address_email(organizer_prop) if organizer_prop else ""
119     organizer_name = _address_name(organizer_prop) if organizer_prop else ""

121     attendees: list[Attendee] = []
122     my_participation = "needs-action"
123     for prop in _as_list(vevent.get("ATTENDEE")):
124         email_addr = _address_email(prop)
125         partstat = _PARTSTAT_TO_LOCAL.get(str(prop.params.get("PARTSTAT", "NEEDS-ACTION")).upper(), "needs-action")
126         attendees.append(Attendee(email=email_addr, name=_address_name(prop), participation=partstat))
127         if my_email and email_addr.lower() == my_email.lower():
128             my_participation = partstat

130     rrule_prop = vevent.get("RRULE")
131     recurrence_rule = rrule_prop.to_ical().decode("ascii") if rrule_prop is not None else None

133     attachments = [
134         a for i, prop in enumerate(_as_list(vevent.get("ATTACH"))) if (a := _attachment_from_prop(prop, i))
135     ]

137     uid = str(vevent.get("UID", "")) or new_uid()

139     # Реальный экспорт целого календаря (в отличие от одиночного iTIP-
140     # приглашения) не несёт METHOD:, зато почти всегда несёт свой STATUS:
141     # на каждой встрече (в реальном файле VK Mail — 154 из 174 событий
142     # TENTATIVE) — раньше это вообще не читалось, и все такие события
143     # тихо превращались в "confirmed".
144     status_prop = str(vevent.get("STATUS", "")).upper()
145     if method == "CANCEL" or status_prop == "CANCELLED":

```

```

146         status = "cancelled"
147     elif status_prop == "TENTATIVE":
148         status = "tentative"
149     else:
150         status = "confirmed"

152     return Event(
153         uid=uid,
154         sequence=int(vevent.get("SEQUENCE", 0)),
155         summary=str(vevent.get("SUMMARY", "")),
156         description=str(vevent.get("DESCRIPTION", "")),
157         location=str(vevent.get("LOCATION", "")),
158         dtstart=dtstart,
159         dtend=dtend,
160         all_day=all_day,
161         recurrence_rule=recurrence_rule,
162         organizer_email=organizer_email,
163         organizer_name=organizer_name,
164         is_organizer=bool(my_email) and bool(organizer_email) and organizer_email.lower() == my_email.lower(),
165         status=status,
166         my_participation=my_participation,
167         attendees=attendees,
168         attachments=attachments,
169         raw_ics=raw_ics,
170     )

173 def build_request_ics(event: Event, organizer_email: str, organizer_name: str) -> bytes:
174     return _build(
175         "REQUEST", event, organizer_email=organizer_email, organizer_name=organizer_name, status="CONFIRMED"
176     )

179 def build_cancel_ics(event: Event, organizer_email: str, organizer_name: str) -> bytes:
180     return _build(
181         "CANCEL", event, organizer_email=organizer_email, organizer_name=organizer_name, status="CANCELLED"
182     )

185 def build_caldav_ics(event: Event, organizer_email: str, organizer_name: str) -> bytes:
186     """Объект календаря для PUT на CalDAV-сервер – это не приглашение по
187     почте (METHOD:REQUEST/CANCEL из RFC 5546), а обычное хранимое событие,
188     поэтому без верхнеуровневого iTIP METHOD (см. RFC 4791 – CalDAV сам по
189     себе не про METHOD, это только для scheduling-сообщений)."""
190     return _build(None, event, organizer_email=organizer_email, organizer_name=organizer_name,
status=event.status.upper())

193 def build_reply_ics(event: Event, attendee_email: str, attendee_name: str, participation: str) -> bytes:
194     cal = _new_calendar("REPLY")
195     vevent = icalendar.Event()
196     vevent.add("uid", event.uid)
197     vevent.add("summary", event.summary)
198     vevent.add("sequence", event.sequence)
199     vevent.add("dtstamp", datetime.now(timezone.utc))
200     vevent.add("dtstart", event.dtstart.date() if event.all_day else event.dtstart)

202     organizer = icalendar.vCalAddress(f"mailto:{event.organizer_email}")
203     if event.organizer_name:
204         organizer.params["CN"] = event.organizer_name
205     vevent.add("organizer", organizer, encode=0)

207     attendee = icalendar.vCalAddress(f"mailto:{attendee_email}")
208     attendee.params["CN"] = attendee_name or attendee_email
209     attendee.params["PARTSTAT"] = _PARTSTAT_TO_ICAL.get(participation, "NEEDS-ACTION")
210     vevent.add("attendee", attendee, encode=0)

212     cal.add_component(vevent)
213     return cal.to_ical()

216 def _build(method: str | None, event: Event, *, organizer_email: str, organizer_name: str, status: str) -> bytes:
217     cal = _new_calendar(method)
218     vevent = icalendar.Event()
219     vevent.add("uid", event.uid)
220     vevent.add("summary", event.summary)
221     if event.description:
222         vevent.add("description", event.description)
223     if event.location:
224         vevent.add("location", event.location)
225     vevent.add("sequence", event.sequence)
226     vevent.add("status", status)
227     vevent.add("dtstamp", datetime.now(timezone.utc))
228     if event.all_day:
229         vevent.add("dtstart", event.dtstart.date())
230     else:
231         vevent.add("dtstart", event.dtstart)
232         vevent.add("dtend", event.dtend)

```

```

234     if event.recurrence_rule:
235         vevent.add("rrule", icalendar.vRecur.from_ical(event.recurrence_rule))

237     organizer = icalendar.vCalAddress(f"mailto:{organizer_email}")
238     organizer.params["CN"] = organizer_name or organizer_email
239     vevent.add("organizer", organizer, encode=0)

241     for attendee in event.attendees:
242         addr = icalendar.vCalAddress(f"mailto:{attendee.email}")
243         addr.params["CN"] = attendee.name or attendee.email
244         addr.params["ROLE"] = "REQ-PARTICIPANT"
245         addr.params["PARTSTAT"] = _PARTSTAT_TO_ICAL.get(attendee.participation, "NEEDS-ACTION")
246         addr.params["RSVP"] = "TRUE" if method == "REQUEST" else "FALSE"
247         vevent.add("attendee", addr, encode=0)

249     for attachment in event.attachments:
250         attach_prop = icalendar.vBinary(attachment.payload)
251         attach_prop.params["FMTTYPE"] = attachment.content_type
252         attach_prop.params["X-FILENAME"] = attachment.filename
253         vevent.add("attach", attach_prop, encode=0)

255     cal.add_component(vevent)
256     return cal.to_ical()

259 def _new_calendar(method: str | None) -> icalendar.Calendar:
260     cal = icalendar.Calendar()
261     cal.add("prodid", "-//RedMail//RU")
262     cal.add("version", "2.0")
263     if method:
264         cal.add("method", method)
265     return cal

268 def _as_list(prop) -> list:
269     if prop is None:
270         return []
271     return prop if isinstance(prop, list) else [prop]

274 def _address_email(prop) -> str:
275     value = str(prop)
276     return value[7:] if value.lower().startswith("mailto:") else value

279 def _address_name(prop) -> str:
280     return str(prop.params.get("CN", "")) if hasattr(prop, "params") else ""

283 def _attachment_from_prop(prop, index: int) -> Attachment | None:
284     # ATTACH бывает и внешней ссылкой (URI), а не встроенным файлом — тогда
285     # icalendar отдаёт не vBinary, а обычную строку/vUri; такие пропускаем
286     # (нечего скачивать/показывать как вложение без похода в сеть).
287     if not isinstance(prop, icalendar.vBinary):
288         return None
289     try:
290         payload = base64.b64decode(prop.to_ical())
291     except (ValueError, TypeError):
292         return None
293     filename = str(prop.params.get("X-FILENAME") or prop.params.get("FILENAME") or f"attachment-{index + 1}")
294     content_type = str(prop.params.get("FMTTYPE", "application/octet-stream"))
295     return Attachment(filename=filename, content_type=content_type, payload=payload)

298 def _to_utc(value) -> datetime:
299     if isinstance(value, datetime):
300         return value.astimezone(timezone.utc) if value.tzinfo else value.replace(tzinfo=timezone.utc)
301     return datetime(value.year, value.month, value.day, tzinfo=timezone.utc)

==== src/redmail/caldav_sync.py (553 строк) ====
1  from __future__ import annotations

3  import urllib.error
4  import urllib.request
5  from dataclasses import dataclass
6  from datetime import datetime, timedelta, timezone
7  from urllib.parse import unquote, urlparse
8  from uuid import uuid4

10 import caldav
11 import requests
12 from caldav.lib.error import AuthorizationError, NotFoundError

14 from redmail import itip
15 from redmail.applog import get_logger

17 _log = get_logger("caldav")
18 from redmail.calendar_store import Event

```

```

20 # Некоторые корпоративные прокси/WAF перед CalDAV-сервером отличают
21 # автоматизированные HTTP-библиотеки от обычных десктопных клиентов именно
22 # по User-Agent и применяют к ним более строгую политику — жалоба
23 # "из Evolution и через веб событие создаётся нормально, а из этого клиента
24 # нет, хотя проверка подключения (чтение) проходит" указывает ровно на такую
25 # избирательную фильтрацию по заголовку, а не на блокировку метода PUT как
26 # такового (раз PUT работает у других клиентов на этом же сервере). Библиотека
27 # caldav по умолчанию подставляет "python-caldav/<версия>" — максимально
28 # узнаваемую сигнатуру скриптовой библиотеки; заменяем её на нейтральную,
29 # не выдающую, что это python-скрипт, но и не выдающую себя за чужой продукт.
30 _CALDAV_USER_AGENT = "redmail-caldav-client/1.0"

33 def _with_connection_retry(func, *args, **kwargs):
34     """Один повтор САМОГО сетевого вызова при ConnectionError ("Remote end
35     closed connection without response" и т.п.) — жалоба: "не
36     синхронизируется календарь, при этом проверка подключения проходит".
37     Проверка подключения — это единственный лёгкий PROPFIND, а сама
38     синхронизация — несколько последовательных запросов (push каждой
39     встречи, потом fetch) через пул соединений requests/caldav; сервер
40     (или прокси/балансировщик перед ним в закрытой корпоративной сети)
41     вполне может закрыть простаивающее keep-alive-соединение между
42     запросами, не ответив на очередной — классический случай устаревшего
43     соединения, который в подавляющем большинстве случаев лечится одним
44     повтором на свежем соединении, а не признак настоящей неполадки с
45     сервером или данными. Обёрнуто вокруг КОНКРЕТНОГО вызова caldav/requests,
46     а не всего метода целиком — иначе к моменту, когда метод сам ловит
47     Exception и заворачивает его в CalDavSyncError, исходный
48     requests.exceptions.ConnectionError уже не различить."""
49     try:
50         return func(*args, **kwargs)
51     except requests.exceptions.ConnectionError:
52         return func(*args, **kwargs)

54 # CalDAV с сервером клиента (VK Mail/Exchange) — сеть закрытая корпоративная,
55 # у самого redmail нет прямого способа её нащупать заранее, поэтому адрес
56 # сервера вводится пользователем вручную в настройках (см. SettingsDialog/
57 # config_store.load_caldav_settings), а не автоопределяется. Логин/пароль —
58 # те же, что для IMAP/SMTP (общий Account), см. MainWindow.on_caldav_sync.
59 #
60 # Двусторонняя синхронизация: события, где мы организатор, отправляются на
61 # сервер (push); всё, что есть на сервере в окне синхронизации, подтягивается
62 # в локальный calendar_store (pull) и связывается по UID (тот же UID, что
63 # использует iTIP-путь через почту — см. itip.py) — событие, полученное
64 # когда-то по приглашению, и то же событие с CalDAV-сервера не задвоятся.

67 @dataclass
68 class CalDavAccount:
69     url: str
70     username: str
71     password: str
72     # "password" — Basic (логин + app-пароль); "kerberos" — SSO по доменному
73     # билету через SPNEGO/Negotiate, как у IMAP/SMTP (Account.auth_type).
74     # Уточнение от пользователя: веб-приложение VK авторизует через keytab,
75     # то есть HTTP-фронт (тот же, что отдаёт CalDAV) принимает Negotiate —
76     # тогда app-пароль (который VK для сторонних клиентов требует и порой
77     # капризно: "Application password is REQUIRED") не нужен вовсе.
78     auth_type: str = "password"
79     # См. imap_client.Account.keytab_path/principal — тот же необязательный
80     # keytab как источник билета для SSO.
81     keytab_path: str = ""
82     principal: str = ""

85 class CalDavSyncError(Exception):
86     """Ошибка подключения к серверу или синхронизации — показывается
87     пользователю как есть, не проглатывается молча (тот же принцип, что и
88     у остальных сетевых операций в проекте)."""

91 def probe_auth_schemes(url: str) -> list[str]:
92     """Делает запрос без учётных данных к CalDAV-серверу и смотрит, какие
93     схемы аутентификации он предлагает в ответе 401 (заголовок
94     WWW-Authenticate) — чтобы понять, есть ли там Kerberos/SPNEGO
95     (Negotiate), а не гадать вслепую по одному лишь "Unauthorized".
96     Реальный повод: тонкий веб-клиент (аналог OWA) у той же почты
97     заходит через SSO, значит Kerberos-инфраструктура в организации
98     есть — но CalDAV в этом приложении сейчас умеет только логин и
99     пароль (Basic), и неясно, предлагает ли сам CalDAV-сервер вообще
100     Negotiate, пока не проверишь напрямую."""
101     req = urllib.request.Request(url, method="PROPFIND", headers={"Depth": "0"})
102     try:
103         urllib.request.urlopen(req, timeout=10)
104         return [] # ответил без 401 вовсе — не тот случай, для которого зовут эту функцию
105     except urllib.error.HTTPError as exc:
106         if exc.code != 401:
107             return []
108         values = exc.headers.get_all("WWW-Authenticate") or []

```

```

109         return [v.split(None, 1)[0] for v in values if v.strip()]
110     except urllib.error.URLError:
111         return [] # диагностика best-effort — не должна маскировать исходную ошибку вызывающего кода

114 def _auth_scheme_hint(url: str) -> str:
115     schemes = probe_auth_schemes(url)
116     if not schemes:
117         return ""
118     if any(s.lower() == "negotiate" for s in schemes):
119         return (
120             f"Сервер предлагает: {'', '.join(schemes)} — включая Negotiate (Kerberos/SPNEGO): "
121             "переключите способ входа учётной записи на Kerberos (SSO) — CalDAV тогда пойдёт "
122             "по доменному билету, без app-пароля."
123         )
124     return f"Сервер предлагает только: {'', '.join(schemes)} — SSO (Negotiate) на этом сервере недоступен."

127 @dataclass
128 class CalDavCalendarInfo:
129     """Один календарь, обнаруженный на сервере при обходе calendar-home-set —
130     задача "настроить" получение расширенных календарей для VK Mail". У VK
131     (и вообще по CalDAV, см. переписку с пользователем) нет делегирования
132     всего аккаунта (calendar-проху, как у Apple/Nextcloud) — есть только
133     пошаренные по отдельности календари, и после того как коллега
134     расшарил календарь и приглашение принято В ВЕБ-ИНТЕРФЕЙСЕ VK (accept-
135     invite по CalDAV не бывает), сервер сам кладёт этот календарь в HASH
136     calendar-home-set как обычную коллекцию — единственное, что остаётся
137     сделать здесь, это её ОБНАРУЖИТЬ и показать, чей это календарь."""

139     url: str
140     name: str
141     owner: str | None
142     is_shared: bool
143     read_only: bool

146 _CALDAV_CALENDAR_TAG = "{urn:ietf:params:xml:ns:caldav}calendar"
147 _CALDAV_HOME_SET_PROP = "{urn:ietf:params:xml:ns:caldav}calendar-home-set"
148 _CALENDAR_DISCOVERY_PROPS = [
149     "{DAV:}resourcetype",
150     "{DAV:}displayname",
151     "{DAV:}owner",
152     "{DAV:}current-user-privilege-set",
153 ]

156 @dataclass
157 class PropfindResult:
158     """Один <response> из multistatus-ответа на PROPFIND: href, статус и
159     свойства (tag → значение: текст, список тегов для resourcetype, список
160     href для ссылочных свойств, сам элемент для сложных вроде
161     current-user-privilege-set)."""

163     href: str
164     status: int
165     properties: dict[str, object]

168 def _propfind_body(props: list[str]) -> str:
169     """Тело PROPFIND-запроса под нужные свойства. Библиотека caldav в
170     разных версиях по-разному принимает список свойств и по-разному
171     отдаёт разобранный ответ (в 3.x — results, в 2.x — только дерево);
172     свой XML на входе и свой разбор дерева на выходе (_parse_multistatus)
173     не зависят от этих различий — нужен только сырой lxml-tree ответа."""
174     from lxml import etree

176     root = etree.Element("{DAV:}propfind", nsmap={"d": "DAV:", "c": "urn:ietf:params:xml:ns:caldav"})
177     prop = etree.SubElement(root, "{DAV:}prop")
178     for name in props:
179         etree.SubElement(prop, name)
180     return etree.tostring(root, xml_declaration=True, encoding="utf-8").decode("utf-8")

183 def _status_code(text: str | None) -> int:
184     for part in (text or "").split():
185         if part.isdigit():
186             return int(part)
187     return 200

190 def _prop_value(element) -> object:
191     children = [child for child in element if isinstance(child.tag, str)]
192     if not children:
193         return (element.text or "").strip()
194     if element.tag == "{DAV:}resourcetype":
195         return [child.tag for child in children]
196     if all(child.tag == "{DAV:}href" for child in children):
197         return [(child.text or "").strip() for child in children]

```

```

198         return element

201 def _parse_multistatus(tree) -> list[_PropfindResult]:
202     """Разбор <multistatus> ответа PROPFIND (RFC 4918): по одному
203     результату на <response>, в properties — только свойства из
204     propstat со статусом 2xx."""
205     results: list[_PropfindResult] = []
206     if tree is None:
207         return results
208     for response in tree.iter("{DAV:}response"):
209         href_el = response.find("{DAV:}href")
210         href = (href_el.text or "").strip() if href_el is not None else ""
211         properties: dict[str, object] = {}
212         statuses: list[int] = []
213         for propstat in response.findall("{DAV:}propstat"):
214             status_el = propstat.find("{DAV:}status")
215             code = _status_code(status_el.text if status_el is not None else None)
216             statuses.append(code)
217             if code // 100 != 2:
218                 continue
219             prop = propstat.find("{DAV:}prop")
220             if prop is None:
221                 continue
222             for element in prop:
223                 if isinstance(element.tag, str):
224                     properties[element.tag] = _prop_value(element)
225         if statuses:
226             status = 200 if any(code // 100 == 2 for code in statuses) else statuses[0]
227         else:
228             direct = response.find("{DAV:}status")
229             status = _status_code(direct.text) if direct is not None else 200
230         results.append(_PropfindResult(href=href, status=status, properties=properties))
231     return results

234 def _href_path(href: str) -> str:
235     """Путь без схемы/хоста и завершающего слэша — owner может прийти и
236     абсолютным URL, и просто путём, сравнивать нужно только путь."""
237     return unquote(urlparse(href).path).rstrip("/")

240 def _has_write_privilege(value: object) -> bool:
241     """current-user-privilege-set не разбирается caldav-библиотекой в
242     удобный список (вложенные <privilege><write/></privilege> не подходят
243     под её общий разбор свойств) — приходит как есть, "сырым" XML-
244     элементом, ищем write/all прямо в нём. Свойство отсутствует вовсе
245     (None) — не считаем календарь урезанным без причины: сервер и так
246     отклонит PUT 403-м, если прав на самом деле нет, а свойство мог просто
247     не отдать (см. переписку: cs:invite/cs:shared-url у VK могут прийти
248     404 — не факт, что current-user-privilege-set при этом тоже не будет)."""
249     if value is None:
250         return True
251     if not hasattr(value, "iter"):
252         return False
253     for child in value.iter():
254         local = child.tag.rsplit(")", 1)[-1] if isinstance(child.tag, str) else ""
255         if local in ("write", "all", "write-content"):
256             return True
257     return False

260 class CalDavSession:
261     """Одно CalDAV-соединение на сессию синхронизации — тот же принцип, что
262     у ImapSession: подключение переиспользуется, а не открывается заново на
263     каждую операцию. Синхронизация запускается вручную (кнопка в
264     календаре), автоматического фоновой опроса нет — сервер ещё ни разу
265     не проверялся вживую, включать это по таймеру было бы преждевременно."""

267     def __init__(self, account: CalDavAccount):
268         self.account = account
269         # Без явного таймута зависший/недоступный сервер (закрытая
270         # корпоративная сеть, где угодно может быть неверно настроенный
271         # прокси/файрвол) мог держать HTTP-запрос сколько угодно — а ведь
272         # on_caldav_sync() до сих пор шёл синхронно в основном потоке
273         # интерфейса: жалоба "после настройки CalDAV сломалась отправка,
274         # просмотр, переход между папками и получение почты" — на деле не
275         # сломалась, а всё это время буквально ждала одного зависшего
276         # сетевого запроса. См. MainWindow.on_caldav_sync — сама
277         # синхронизация теперь выполняется в фоновом потоке, но таймаут
278         # всё равно нужен: без него поток просто завис бы бесконечно вместо
279         # того, чтобы сообщить об ошибке.
280         client_kwargs: dict = {"timeout": 30, "headers": {"User-Agent": _CALDAV_USER_AGENT}}
281         try:
282             if account.auth_type == "kerberos":
283                 # Импорт внутри — см. gssapi_sasl.py: requests_gssapi тянет
284                 # системные библиотеки Kerberos, которых нет там, где SSO не
285                 # используется (и на машине для тестов); обычный пароль от
286                 # этого ломаться не должен.

```



```

287         import requests_gssapi
288
289         from redmail import gssapi_sasl
290
291         # mutual_authentication=OPTIONAL, а не REQUIRED по умолчанию:
292         # за корпоративным reverse-проxy/балансировщиком ответ сервера
293         # часто приходит без встречного GSSAPI-токена, и строгая
294         # взаимная проверка роняла бы уже успешно прошедший вход.
295         # creds=None – билет из системного кэша; с keytab – из него.
296         client_kwargs["auth"] = requests_gssapi.HTTPSPNEGOAuth(
297             mutual_authentication=requests_gssapi.OPTIONAL,
298             creds=gssapi_sasl.acquire_credentials(account.keytab_path, account.principal),
299         )
300     else:
301         client_kwargs["username"] = account.username
302         client_kwargs["password"] = account.password
303     self._client = caldav.DAVClient(account.url, **client_kwargs)
304     # Всегда обычный requests.Session вместо того, что выбрал caldav:
305     # при наличии niquests он берёт его, а niquests по умолчанию
306     # договаривается об HTTP/2 через ALPN. Жалоба (и после смены
307     # User-Agent, и после прямого URL календаря): чтение работает,
308     # а PUT события рвётся "Remote end closed connection without
309     # response" – при этом Evolution (libsoup, HTTP/1.1) на том же
310     # сервере события создаёт. Обрыв запроса с телом без единого
311     # байта ответа при рабочих запросах без тела – характерный
312     # симптом прокси/балансировщика с неполной поддержкой HTTP/2
313     # для методов с телом; обычный requests ходит по HTTP/1.1, как
314     # Evolution. Заодно с ним документирована и отлажена связка с
315     # HTTPSPNEGOAuth (хук на 401 через r.connection/r.request.copy()).
316     try:
317         self._client.session.close()
318     except Exception:
319         pass
320     self._client.session = requests.Session()
321 except Exception as exc:
322     _log.error("CalDAV %s: не удалось создать соединение (%s): %s", account.url, account.auth_type, exc)
323     raise CalDavSyncError(f"Не удалось создать CalDAV-соединение: {exc}") from exc
324 self._calendar = None
325 _log.info("CalDAV %s: соединение создано (%s, %s)", account.url, account.username, account.auth_type)
326
327 def _primary_calendar(self):
328     if self._calendar is None:
329         # account.url – это URL КОНКРЕТНОГО календаря (см.
330         # list_calendars_detailed для обнаружения таких URL, включая
331         # расшаренные другими пользователями), обращаемся к нему
332         # напрямую. Раньше здесь было principal.calendars()[0] –
333         # дисквери первого попавшегося календаря аккаунта НЕЗАВИСИМО
334         # от того, какой именно caldav_url был настроен: из-за этого
335         # ЛЮБОЙ второй настроенный CalDAV-календарь того же аккаунта
336         # молча синхронизировался бы с тем же самым первым календарём,
337         # что и делало поддержку нескольких/расшаренных календарей
338         # бессмысленной – без этого исправления задача "получение
339         # расшаренных календарей для VK Mail" попросту не работала бы.
340         self._calendar = caldav.Calendar(client=self._client, url=self.account.url)
341     return self._calendar
342
343 def list_calendar_names(self) -> list[str]:
344     try:
345         principal = self._client.principal()
346         return [cal.get_display_name() or str(cal.url) for cal in _with_connection_retry(principal.calendars)]
347     except AuthorizationError as exc:
348         raise CalDavSyncError(f"Не удалось получить список календарей:
349 {exc}.({_auth_scheme_hint(self.account.url)})") from exc
350     except Exception as exc:
351         raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc
352
353 def list_calendars_detailed(self) -> list[CalDavCalendarInfo]:
354     """Все календари, включая расшаренные нам коллегами (не только
355     собственные) – owner коллекции указывает на ЧУЖОЙ принципал, если
356     это так. См. CalDavCalendarInfo – задача "настроить получение
357     расшаренных календарей для VK Mail".
358
359     Жалоба после первой версии: "поиск находит основной, но не находит
360     расшаренный другого пользователя", при том что Evolution на том же
361     сервере его видит. Два отличия от простого PROPFIND Depth:1 по
362     одному calendar-home-set, которые Evolution (e-webdav-discover)
363     делает, а первая версия – нет: (1) calendar-home-set у принципала
364     может содержать НЕСКОЛЬКО href (свои календари в одном доме, чужие
365     расшаренные – в другом), caldav-библиотека берёт только первый;
366     (2) внутри дома могут лежать обычные под-коллекции (например,
367     shared/), в которых уже лежат календари – Depth:1 их не видит,
368     Evolution в такие коллекции спускается."""
369     try:
370         principal = self._client.principal()
371         home_urls = self._calendar_home_urls(principal)
372     except AuthorizationError as exc:
373         raise CalDavSyncError(f"Не удалось получить список календарей:
374 {exc}.({_auth_scheme_hint(self.account.url)})") from exc
375     except Exception as exc:

```

```

374         raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc

376     my_principal_path = _href_path(str(principal.url))
377     infos: list[CalDavCalendarInfo] = []
378     seen: set[str] = set()
379     for home_url in home_urls:
380         self._collect_calendars(home_url, my_principal_path, infos, seen, depth_left=2)
381     _log.info("CalDAV %s: домов календарей %d, найдено календарей %d (расшаренных %d)",
382             self.account.url, len(home_urls), len(infos), sum(1 for i in infos if i.is_shared))
383     return infos

385     def _calendar_home_urls(self, principal) -> list[str]:
386         """Все href из calendar-home-set принципала (не только первый)."""
387         urls: list[str] = []
388         try:
389             response = _with_connection_retry(
390                 self._client.propfind, str(principal.url), _propfind_body([_CALDAV_HOME_SET_PROP]), 0
391             )
392             for result in _parse_multistatus(response.tree):
393                 value = result.properties.get(_CALDAV_HOME_SET_PROP)
394                 hrefs = value if isinstance(value, list) else [value]
395                 for href in hrefs:
396                     if isinstance(href, str) and href:
397                         urls.append(str(self._client.url.join(href)))
398         except Exception:
399             pass # ниже - обычный путь библиотеки, как раньше
400         if not urls:
401             urls.append(str(principal.calendar_home_set.url))
402         return urls

404     def _collect_calendars(
405         self, url: str, my_principal_path: str, infos: list[CalDavCalendarInfo], seen: set[str], depth_left: int
406     ) -> None:
407         try:
408             response = _with_connection_retry(
409                 self._client.propfind, url, _propfind_body(_CALENDAR_DISCOVERY_PROPS), 1
410             )
411         except AuthorizationError as exc:
412             raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc
413         except Exception as exc:
414             raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc
415         return # вложенная коллекция не отвечает - не роняем весь список
416     own_path = _href_path(url)
417     for result in _parse_multistatus(response.tree):
418         if result.status not in (200, 207):
419             continue
420         resource_type = result.properties.get("{DAV:}resourcetype") or []
421         tags = [str(t) for t in (resource_type if isinstance(resource_type, list) else [resource_type])]
422         full_url = str(self._client.url.join(result.href))
423         path = _href_path(full_url)
424         if path == own_path:
425             continue # сама необходимая коллекция
426         # Точное имя тега, не подстрока "calendar": у расшаренных
427         # коллекций в resource_type бывает {http://calendarserver.org/ns/}shared
428         # - "calendarserver" содержит "calendar", ложное срабатывание.
429         is_calendar = _CALDAV_CALENDAR_TAG in tags
430         is_collection = any(t.endswith("}collection") for t in tags)
431         if is_calendar:
432             if path in seen:
433                 continue
434             seen.add(path)
435             owner_raw = result.properties.get("{DAV:}owner")
436             if isinstance(owner_raw, list):
437                 owner_raw = owner_raw[0] if owner_raw else None
438             owner = owner_raw if isinstance(owner_raw, str) and owner_raw else None
439             infos.append(CalDavCalendarInfo(
440                 url=full_url,
441                 name=result.properties.get("{DAV:}displayname") or result.href,
442                 owner=owner,
443                 is_shared=owner is not None and _href_path(owner) != my_principal_path,
444                 read_only=not _has_write_privilege(result.properties.get("{DAV:}current-user-privilege-set")),
445             ))
446         elif is_collection and depth_left > 1:
447             self._collect_calendars(full_url, my_principal_path, infos, seen, depth_left - 1)

450     def fetch_events(self, start: datetime, end: datetime, my_email: str) -> list[Event]:
451         """События сервера в окне [start, end). Разбор VEVENT переиспользует
452         itip.parse_ics_events - ту же логику, что уже проверена на реальных
453         .ics-вложениях и импорте, вместо второго независимого парсера
454         одного и того же формата."""
455         calendar = self._primary_calendar()
456         try:
457             results = _with_connection_retry(calendar.date_search, start, end)
458         except Exception as exc:
459             _log.error("CalDAV %s: получение событий не удалось: %s", self.account.url, exc)
460             raise CalDavSyncError(f"Не удалось получить события с сервера: {exc}") from exc

```

```

462     events: list[Event] = []
463     for obj in results:
464         try:
465             raw = obj.data
466             raw_bytes = raw.encode("utf-8") if isinstance(raw, str) else raw
467             events.extend(itip.parse_ics_events(raw_bytes, my_email))
468         except Exception:
469             continue # одно повреждённое/непонятное событие не должно валить всю синхронизацию
470     _log.info("CalDAV %s: получено объектов %d, событий %d (окно %s - %s)",
471             self.account.url, len(results), len(events), start.date(), end.date())
472     return events

474 def push_event(self, event: Event, organizer_email: str, organizer_name: str) -> None:
475     """Создаёт событие на сервере или обновляет уже существующее
476     (находит по UID - тот же UID, что уже используется локально)."""
477     calendar = self._primary_calendar()
478     ics_text = itip.build_caldav_ics(event, organizer_email, organizer_name).decode("utf-8")
479     try:
480         existing = _with_connection_retry(calendar.event_by_uid, event.uid)
481     except NotFoundError:
482         existing = None
483     except Exception as exc:
484         raise CalDavSyncError(f"Не удалось проверить событие на сервере: {exc}") from exc

486     try:
487         if existing is not None:
488             existing.data = ics_text
489             _with_connection_retry(existing.save)
490             _log.info("CalDAV %s: событие обновлено uid=%s", self.account.url, event.uid)
491         else:
492             _with_connection_retry(calendar.save_event, ics_text)
493             _log.info("CalDAV %s: событие создано uid=%s", self.account.url, event.uid)
494     except Exception as exc:
495         _log.error("CalDAV %s: сохранение события uid=%s не удалось: %s", self.account.url, event.uid, exc)
496         raise CalDavSyncError(f"Не удалось сохранить событие на сервере: {exc}") from exc

498 def test_write_access(self) -> None:
499     """Настоящая проверка записи: создаёт одноразовое тестовое событие на
500     сервере и сразу удаляет его. В отличие от одной лишь проверки чтения
501     (PROPFIND/список календарей), это ловит именно ту ситуацию, из-за
502     которой возникла путаница - "проверка подключения проходит, а
503     синхронизация нет": проверка подключения раньше проверяла только
504     чтение, поэтому расхождение между чтением и записью (например,
505     избирательная фильтрация PUT на стороне сети по каким-то признакам
506     запроса) не обнаруживалось заранее, а всплывало только при реальной
507     синхронизации. Бросает CalDavSyncError с текстом самой сетевой
508     ошибки, если запись не удалась - чтение при этом может быть исправно."""
509     calendar = self._primary_calendar()
510     test_uid = f"redmail-conntest-{uuid4()}@redmail"
511     start = datetime.now(timezone.utc).replace(microsecond=0)
512     test_event = Event(
513         uid=test_uid,
514         summary="redmail: проверка подключения (можно удалить)",
515         dtstart=start,
516         dtend=start + timedelta(minutes=1),
517         organizer_email=self.account.username,
518         organizer_name=self.account.username,
519         is_organizer=True,
520     )
521     ics_text = itip.build_caldav_ics(test_event, test_event.organizer_email,
test_event.organizer_name).decode("utf-8")
522     try:
523         _with_connection_retry(calendar.save_event, ics_text)
524     except Exception as exc:
525         _log.error("CalDAV %s: проверка записи не удалась: %s", self.account.url, exc)
526         raise CalDavSyncError(f"Запись на сервер не удалась: {exc}") from exc
527     _log.info("CalDAV %s: проверка записи прошла", self.account.url)
528     try:
529         existing = _with_connection_retry(calendar.event_by_uid, test_uid)
530         _with_connection_retry(existing.delete)
531     except Exception:
532         pass # уборка тестового события - не критично, если не получилось

534 def delete_event(self, uid: str) -> None:
535     calendar = self._primary_calendar()
536     try:
537         existing = _with_connection_retry(calendar.event_by_uid, uid)
538     except NotFoundError:
539         return # уже нет на сервере - нечего удалять, не ошибка
540     except Exception as exc:
541         raise CalDavSyncError(f"Не удалось найти событие на сервере: {exc}") from exc
542     try:
543         _with_connection_retry(existing.delete)
544         _log.info("CalDAV %s: событие удалено uid=%s", self.account.url, uid)
545     except Exception as exc:
546         _log.error("CalDAV %s: удаление события uid=%s не удалось: %s", self.account.url, uid, exc)
547         raise CalDavSyncError(f"Не удалось удалить событие на сервере: {exc}") from exc

549 def close(self) -> None:

```

```

550         try:
551             self._client.close()
552         except Exception:
553             pass

===== src/redmail/archive_store.py (448 строк, показаны первые 232) =====
1  from __future__ import annotations

3  import mailbox
4  import sqlite3
5  from contextlib import closing
6  from email import message_from_bytes
7  from email.header import decode_header
8  from email.message import EmailMessage, Message
9  from email.utils import formatdate, parseaddr
10 from pathlib import Path

12 from redmail.imap_client import decode_bytes_safely, MessageContent, MessageSummary, extract_content,
parse_importance

14 # Свой формат: один файл SQLite на архив. Не PST — по назначению аналог
15 # (папка с письмами, которую можно отключить от сервера и открыть локально),
16 # но не побайтово совместим с ним. Реальный .pst слишком сложен, чтобы
17 # писать самим (см. обсуждение — читаем чужие через rufpf, свои не пишем).
18 _FORMAT_VERSION = 1

20 _SCHEMA = """
21 CREATE TABLE IF NOT EXISTS meta (
22     key TEXT PRIMARY KEY,
23     value TEXT NOT NULL
24 );

26 CREATE TABLE IF NOT EXISTS messages (
27     id INTEGER PRIMARY KEY AUTOINCREMENT,
28     folder TEXT NOT NULL,
29     subject TEXT NOT NULL,
30     sender TEXT NOT NULL,
31     sender_email TEXT NOT NULL,
32     date TEXT NOT NULL,
33     message_id TEXT NOT NULL,
34     has_attachments INTEGER NOT NULL DEFAULT 0,
35     marker_color TEXT,
36     importance TEXT NOT NULL DEFAULT 'normal',
37     raw BLOB NOT NULL
38 );
39 """

42 class NotAnArchiveError(Exception):
43     """Файл существует, но это не наш формат архива (или он новее, чем мы умеем читать)."""

46 def _connect(path: Path) -> sqlite3.Connection:
47     conn = sqlite3.connect(path)
48     conn.executescript(_SCHEMA)
49     return conn

52 def create_archive(path: Path) -> None:
53     """Создаёт пустой файл архива. Не трогает уже существующий по этому пути."""
54     path.parent.mkdir(parents=True, exist_ok=True)
55     with closing(_connect(path)) as conn:
56         row = conn.execute("SELECT value FROM meta WHERE key = 'format_version'").fetchone()
57         if row is None:
58             conn.execute(
59                 "INSERT INTO meta (key, value) VALUES ('format_version', ?)", (str(_FORMAT_VERSION),)
60             )
61             conn.commit()

64 def is_archive_file(path: Path) -> bool:
65     """Дешёвая проверка перед тем, как предлагать открыть файл как архив."""
66     if not path.exists():
67         return False
68     try:
69         with closing(sqlite3.connect(path)) as conn:
70             row = conn.execute("SELECT value FROM meta WHERE key = 'format_version'").fetchone()
71     except sqlite3.DatabaseError:
72         return False
73     return row is not None

76 def list_folders(path: Path) -> list[str]:
77     with closing(_connect(path)) as conn:
78         rows = conn.execute("SELECT DISTINCT folder FROM messages ORDER BY folder").fetchall()
79         return [row[0] for row in rows]

82 def rename_folder(path: Path, old_name: str, new_name: str) -> None:

```

```

83     # У архива нет отдельной таблицы папок — "папка" это просто текстовое
84     # поле у писем (см. схему выше), так что переименование — это
85     # переименование значения этого поля у всех писем сразу. Если
86     # new_name совпадает с уже существующей папкой, письма просто
87     # сольются в неё — это ожидаемо, отдельно не запрещаем.
88     #
89     # Дерево в интерфейсе строится из этих строк разбиением по "/" — у
90     # промежуточного узла ("Работа" для писем в "Работа/Проекты") нет
91     # отдельной записи, только у его вложенных подпапок. Чтобы
92     # переименование такого узла вообще что-то делало, переименовываем и
93     # все вложенные "old_name/..." тем же префиксом.
94     with closing(_connect(path)) as conn:
95         conn.execute("UPDATE messages SET folder = ? WHERE folder = ?", (new_name, old_name))
96         conn.execute(
97             "UPDATE messages SET folder = ? || substr(folder, ?) WHERE folder LIKE ? ESCAPE '\\'",
98             (new_name, len(old_name) + 1, old_name.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_") +
"/%"),
99         )
100         conn.commit()

103 def list_messages(path: Path, folder: str) -> list[MessageSummary]:
104     with closing(_connect(path)) as conn:
105         rows = conn.execute(
106             "SELECT id, subject, sender, sender_email, date, message_id, has_attachments, marker_color, importance
"
107             "FROM messages WHERE folder = ? ORDER BY date DESC, id DESC",
108             (folder,),
109         ).fetchall()
110     return [
111         MessageSummary(
112             uid=row[0], subject=row[1], sender=row[2], sender_email=row[3], date=row[4], message_id=row[5],
113             has_attachments=bool(row[6]), marker_color=row[7], importance=row[8],
114             is_read=True, # архив — уже разобранный почта, не показываем "непрочитанным" вечно
115         )
116         for row in rows
117     ]

120 def get_message_content(path: Path, message_id: int) -> MessageContent:
121     return extract_content(message_from_bytes(get_message_raw(path, message_id)))

124 def get_message_raw(path: Path, message_id: int) -> bytes:
125     with closing(_connect(path)) as conn:
126         row = conn.execute("SELECT raw FROM messages WHERE id = ?", (message_id,)).fetchone()
127     if row is None:
128         raise KeyError(f"В архиве нет письма с id={message_id}")
129     return row[0]

132 def set_marker(path: Path, message_id: int, color: str | None) -> None:
133     with closing(_connect(path)) as conn:
134         conn.execute("UPDATE messages SET marker_color = ? WHERE id = ?", (color, message_id))
135     conn.commit()

138 def delete_messages(path: Path, message_ids: list[int]) -> None:
139     if not message_ids:
140         return
141     placeholders = ",".join("?" * len(message_ids))
142     with closing(_connect(path)) as conn:
143         conn.execute(f"DELETE FROM messages WHERE id IN ({placeholders})", message_ids)
144     conn.commit()

147 def append_raw_message(path: Path, folder: str, raw: bytes) -> int:
148     """Разбирает raw (полное письмо в формате RFC 822) и добавляет в архив."""
149     with closing(_connect(path)) as conn:
150         row_id = _insert_raw(conn, folder, raw)
151         conn.commit()
152     return row_id

155 def _insert_raw(conn: sqlite3.Connection, folder: str, raw: bytes) -> int:
156     message = message_from_bytes(raw)
157     sender_display, sender_email = _decode_from(message)
158     content = extract_content(message)
159     cursor = conn.execute(
160         "INSERT INTO messages (folder, subject, sender, sender_email, date, message_id, "
161         "has_attachments, marker_color, importance, raw) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
162         (
163             folder,
164             _decode_mime_words(message.get("Subject")) or "(без темы)",
165             sender_display,
166             sender_email,
167             _format_date(message.get("Date")),
168             message.get("Message-Id", "") or "",
169             int(bool(content.attachments)),

```

```

170         None,
171         parse_importance(message),
172         raw,
173     ),
174 )
175 return cursor.lastrowid

178 def _decode_mime_words(value: str | None) -> str:
179     if not value:
180         return ""
181     try:
182         parts = decode_header(value)
183     except Exception:
184         # decode_header() поднимает HeaderParseError на действительно
185         # повреждённом base64/QP (не только на пустой строке) – реальный
186         # случай: PST-свойство "имя отправителя" иногда хранит RFC2047-имя
187         # ОБРЕЗАННЫМ (см. _pst_message_to_raw), и обрезка иногда режет
188         # прямо посреди base64-группы, оставляя невалидные символы. Без
189         # этого один такой отправитель падал бы всем импортом .pst разом
190         # (жалоба: "загрузка из pst все ещё некорректно загружает...
191         # авторов") – лучше показать значение как есть, чем потерять
192         # письмо или весь импорт целиком.
193         return value
194     return "".join(
195         decode_bytes_safely(chunk, encoding) if isinstance(chunk, bytes) else chunk
196         for chunk, encoding in parts
197     )

200 def _decode_from(message: Message) -> tuple[str, str]:
201     name, addr = parseaddr(message.get("From", ""))
202     decoded_name = _decode_mime_words(name)
203     if decoded_name:
204         return decoded_name, addr
205     # Если в "From" нет реального адреса в <угловых скобках> – только
206     # закодированное отображаемое имя целиком – parseaddr() кладёт всю
207     # строку в "адрес" (RFC 822 не делит её на имя+адрес без <>), и она
208     # так и остаётся закодированной как =?utf-8?b?...?=?, потому что
209     # decode_header() выше применялся только к "имени". Найдено на
210     # реальном PST (жалоба: "не распознали адреса отправителей в архиве").
211     decoded_addr = _decode_mime_words(addr)
212     if decoded_addr != addr:
213         # "Адрес" сам оказался закодированным именем, не настоящим
214         # e-mail – реального адреса в письме не было.
215         return decoded_addr or "(неизвестно)", ""
216     return addr or "(неизвестно)", addr

219 def _format_date(raw_date: str | None) -> str:
220     if not raw_date:
221         return ""
222     from email.utils import parsedate_to_datetime

224     try:
225         parsed = parsedate_to_datetime(raw_date)
226     except (TypeError, ValueError):
227         return ""
228     if parsed is None:
229         return ""
230     return parsed.strftime("%Y-%m-%d %H:%M")

```